
On the origin of neural scaling laws: from random graphs to natural language

Maissam Barkeshli^{*12} Alberto Alfarano^{*13} Andrey Gromov¹

Abstract

Scaling laws have played a major role in modern AI, providing predictive power over how model performance will improve with increasing resources. This has spurred intense interest in their origin, with a common suggestion being that they arise from power laws already present in the data. Here we study scaling laws for transformers trained to predict random walks on graphs with tunable complexity. We show that this simplified setting already yields scaling laws even in the absence of power laws in the data correlations. We further consider dialing down the complexity of language by training on sequences sampled from increasingly simplified generative language models, from 4,2,1-layer transformer language models down to language bigrams, revealing a monotonic evolution of the scaling exponents. Our results also include scaling laws obtained from training on random walks on random graphs drawn from Erdős-Renyi and scale-free Barabási-Albert ensembles. Finally, we revisit scaling laws for language modeling, demonstrating that several essential results can be reproduced using 2 layer transformers with context length of 100, demonstrate an alternative method for obtaining compute optimal curves, and provide preliminary evidence that maximal update parameterization may be more parameter efficient than standard parameterization.

1. Introduction

Neural scaling laws (Cortes et al., 1993; Hestness et al., 2017; Kaplan et al., 2020; Henighan et al., 2020; Hoffmann et al., 2022) demonstrate that across many vision and natural language tasks, the test loss decreases predictably as a simple power law over many orders of magnitude in number of model parameters N , dataset size D , and compute C . This discovery has had significant impact in practice for language model pretraining, as it allows practitioners to determine how to optimally scale the model and dataset size with compute (Kaplan et al., 2020; Hoffmann et al., 2022; Chowdhery et al., 2022; Grattafiori & et al., 2024; Yang & et al., 2024; 2025; Liu & et al., 2024; Jiang & et al., 2024; Tian et al., 2025). It also provides a way to benchmark breakthroughs in architectures, optimization, and data.

These empirical results have led to significant theoretical work in trying to understand the origin of neural scaling laws. Specifically, why is there a power law decrease in the test loss over many orders of magnitude in N , D , and C , and what sets the exponents? A clear answer to this question may be of significant practical value, since if we understand what sets the exponents, we might understand the extent to which they can be increased, thus increasing the asymptotic efficiency of deep learning methods.

A popular suggestion has been that the power law scaling in the test loss originates from power laws already present in the data. For example, it is well-known that the frequency of words in a corpus of text follows Zipf’s law, with many other power laws having also been characterized in natural language corpora (Piantadosi, 2014; Altmann & Gerlach, 2016). Natural images also exhibit power laws in their spectra (Ruderman, 1994; Maloney et al., 2022). Many theoretical works have shown that in linear or kernel regression, power laws in the test loss do originate from power laws in the data (or in features defined in terms of the data) (Bordelon et al., 2020; Bahri et al., 2021; Spigler et al., 2020; Maloney et al., 2022; Lin et al., 2024; Paquette et al., 2024; Bordelon et al., 2024). From another perspective, if models need to learn a discrete set of tasks to achieve a particular value of test loss, and if these tasks are distributed with power law weighting, then a power law in the test loss follows (Michaud et al., 2024; Ren et al.,

^{*}Equal contribution ¹Meta Superintelligence Labs, FAIR
²Department of Physics, University of Maryland, College Park
and Joint Quantum Institute ³Axiom Math. Correspondence
to: Maissam Barkeshli <maissam@umd.edu>, Alberto Alfarano
<alberto@axiommath.ai>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

2025). The above theories based on linear models with mean square error (MSE) loss are rather far from the setting of auto-regressive sequence modeling with cross-entropy loss. Consequently it is not clear to what extent they are representative of the scaling laws seen in language modeling. A potentially fruitful approach would be to study sequence modeling with data of *tunable complexity*, where one can systematically dial down from a realistic to a highly simplified limit, and track the change in the scaling laws. An additional benefit of datasets with tunable complexity is that they could potentially allow more appropriate comparisons of models across scales: a proper comparison of a small model to a large model should also appropriately increase the complexity of the dataset. Complexity of the dataset is not measured simply in terms of size, but also in terms of the degree of correlations, and hierarchical, compositional structure (Cagnetta et al., 2024).

To this end, we propose to study sequence modeling of random walks on graphs. Graphs and their generalizations – hypergraphs and hierarchical graphical structures – can capture a large portion of many kinds of data, including language, games, and formal mathematics. In particular, walks on graphs can correspond to generating from n -gram models (Sec. 3), providing a simplified model of language. At a more abstract level, walks on graphs can be used to model stepwise inference and chain of thought reasoning (Khona et al., 2024; Besta et al., 2024).

1.1. Our contributions

- We demonstrate that transformers trained to perform next-token prediction on random walks from random graphs, such as Erdős-Renyi and Barabási-Albert graphs, exhibit scaling laws. We empirically demonstrate the first example of scaling laws even when the input data has no power law structures at all (Fig. 1).
- We systematically reduce the complexity of the language setting by training on progressively simpler generative models of language, measuring the scaling laws at each level of complexity (Fig. 2).
- We critically revisit the original scaling law results for LLMs and (i) argue that the best fits are one-dimensional fits of the form shown in Eq. 1, in contrast to certain two-dimensional parametric fits that are common in the literature, and (ii) demonstrate a neural network regression method to obtaining compute optimal scaling laws, which is different from methods presented in the literature to date, and which gives higher accuracy for fitting loss curves.
- We demonstrate that the scaling laws for language found in prior work can be accounted for entirely within a 2-layer transformer with a context length of 100. We show in our experiments that the discrepancy between the Kaplan

and Chinchilla scaling laws can be recovered in terms of whether embedding parameters are included (Porian et al., 2024; Pearce & Song, 2024). We also show preliminary results that maximal update parameterization (Yang & Hu, 2021) might be more parameter-efficient than standard parameterization, which would imply that compute-optimal scaling with μP is not at constant number of tokens per parameter.

Roadmap. In Sec. 2 we provide a review of neural scaling laws and summarize the empirical methodology we use in this paper. In Sec. 3, we discuss our results for learning random walks on random graphs, exhibiting the existence of scaling laws without power laws structures in the data. In Sec. 4, we consider a sequence of datasets of increasing complexity (decreasing entropy), from language bigrams to natural language, and find a monotonic evolution in the scaling exponents. Finally, in Sec. 5, we revisit scaling laws in natural language. We critique some of the common methodology in the literature, discussing the importance of the irreducible loss, shortcomings of some commonly used scaling law formulae, and present our preliminary results for improved parameter efficiency of μP .

2. Neural scaling laws

We focus on scaling laws in the test loss $L(N, D)$, where N is the number of model parameters (sometimes excluding embedding parameters) and D is the dataset size (in tokens). $L(N, D)$ is the optimal loss over a hyperparameter sweep: $L(N, D) = \min_{\{\xi\}} L(N, D; \{\xi\})$, where $\{\xi\}$ denote optimization hyperparameters such as learning rate, batch size, weight decay, etc. Various architectural details, such as aspect ratio, number of attention heads, and so on, are sometimes ignored or held fixed, depending on the study. We denote $L(N)_D$ to be the test loss as a function of N with D held fixed and analogously for $L(D)_N$. Empirically, the test loss fits well to a power law:

$$L(N)_D = E_D + A_D N^{-\alpha_D}, \quad L(D)_N = E_N + B_N D^{-\beta_N} \quad (1)$$

for sufficiently large N or D , respectively. Note that all parameters depend on the variable that is held fixed. In some cases in the literature, the offsets E_D and E_N are not included, which leads to significant underestimation of the exponents α_D and β_N (see Sec. 5).

It is common to consider two-dimensional parametric fits of the form (Hoffmann et al., 2022; Bhagia et al., 2025; Muennighoff et al., 2023; Gadre et al., 2024; Zhang et al., 2024; Kang et al., 2025)

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2)$$

A wide variety of other two-dimensional parametric formulae have been considered; see Table 2 of (Li et al., 2024).

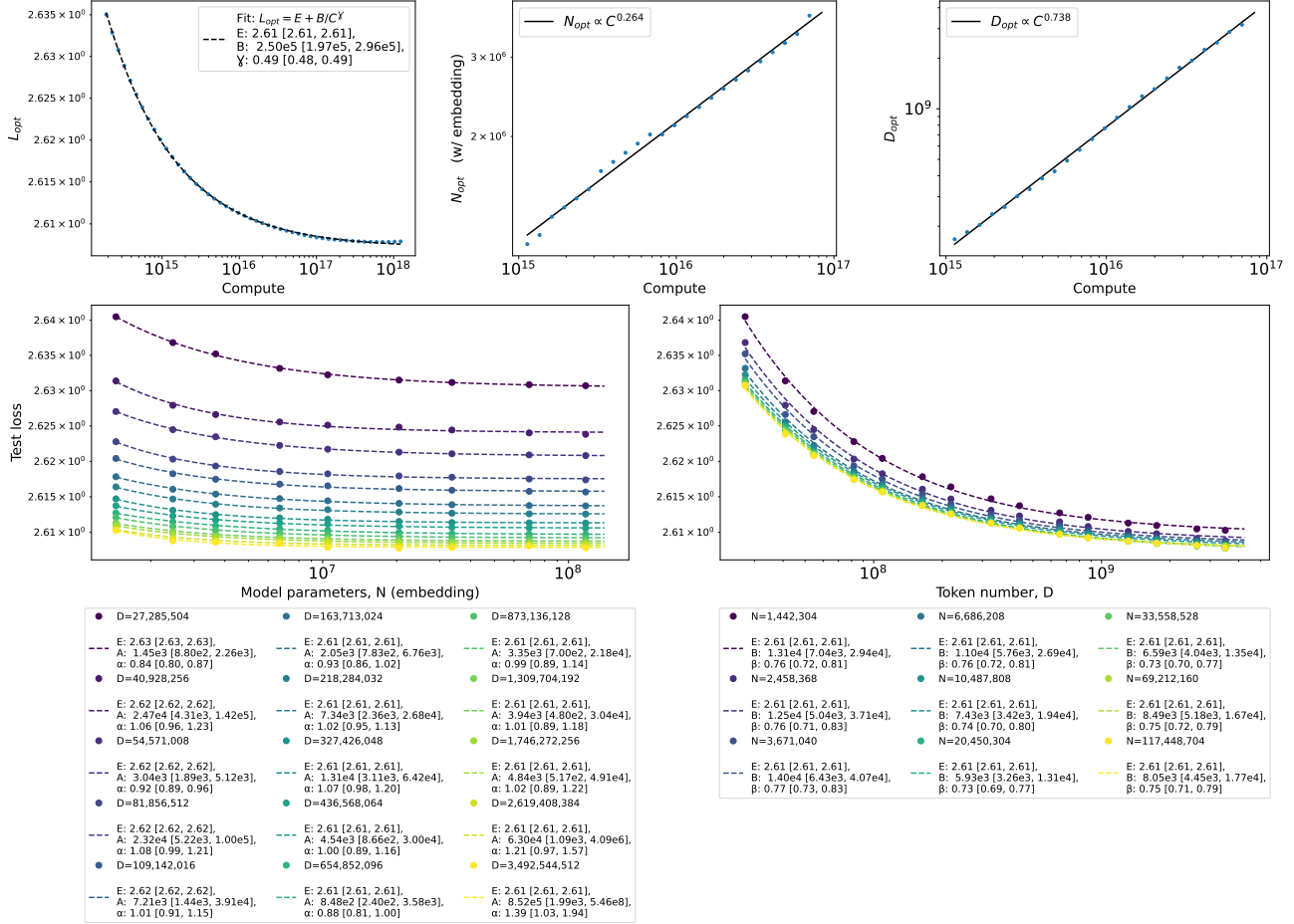


Figure 1. Scaling laws for 2-layer transformers trained on next token prediction on unbiased random walks on an Erdős-Renyi graph, with 8K nodes and 50K edges. Neither the random walks nor the graph exhibits any power laws. Data in bottom two plots are fit to Eq. 1. Mean $\overline{\alpha_D} = 1.028$ with standard deviation 0.129. Mean $\overline{\beta_N} = 0.749$ with standard deviation 0.014. Average MSE for $L(N)_D$ 1d power law fits is 1.07×10^{-8} , compared to 7.03×10^{-8} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits is 8.86×10^{-8} , compared to 2.13×10^{-6} for best exponential fit. Brackets indicate 95% confidence intervals obtained from bias-corrected and accelerated bootstrap method (see Appendix C for details).

The compute in FLOPs (floating point operations) can be expressed exactly in terms of N , D , and number of training steps (Kaplan et al., 2020). It approximately follows the formula $C(N, D) \approx 6NBS$, where S is the number of steps and B is the batch size. For the standard case of 1 epoch training, which we consider hereafter, $D = BS$ and $C(N, D) \approx 6ND$. The compute optimal loss is given by minimizing the loss over N , D subject to fixed compute:

$$L_{\text{opt}}(C) = \min_{N, D | \text{fixed } C} L(N, D). \quad (3)$$

The optimal N and D for a fixed compute are given by

$$N_{\text{opt}} := \operatorname{argmin}_N L(N, D(C, N)), \quad (4)$$

$$D_{\text{opt}} := \operatorname{argmin}_D L(N(D, C), N), \quad (5)$$

where in the equation above $N(D, C) \approx C/6D$ refers to N viewed as a function of D and C , and analogously for

$D(N, C)$. L_{opt} , N_{opt} , and D_{opt} are empirically fit well to

$$L_{\text{opt}} = E_C + KC^{-\gamma}, \quad N_{\text{opt}} = N_0 C^a, \quad D_{\text{opt}} = D_0 C^b. \quad (6)$$

2.1. Methodology

We first fit our results for the loss to the 1d parametric fits, Eq. 1. For details regarding the fitting procedure, see Appendix C. Note that it is crucial to include the irreducible loss E without which one can easily underestimate scaling exponents by over $2\times$ in practice (see Sec. 5).

An important discussion is to what extent the power law fits are accurate (Clauset et al., 2009), especially for extrapolation out to several orders of magnitude in N , D , or C beyond which there is data for $L(N, D)$.¹ In this paper, to

¹Note that the number of orders of magnitude covered by N

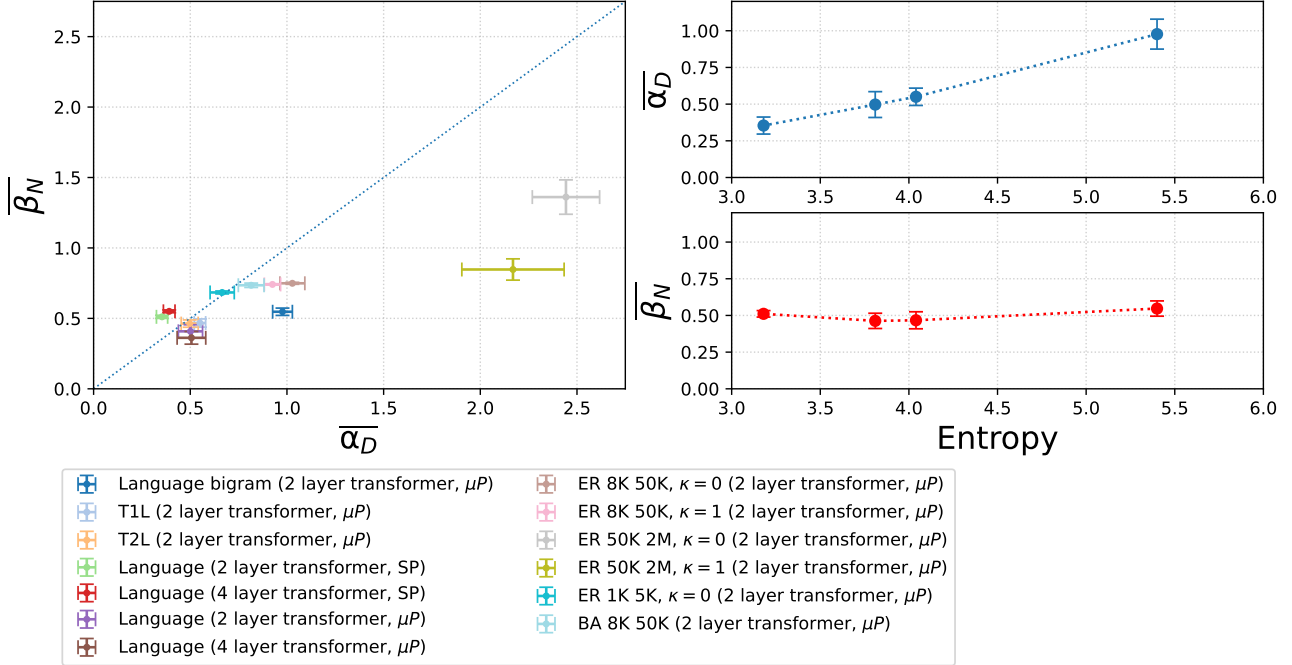


Figure 2. **Left:** Mean exponents $\overline{\alpha_D}$ and $\overline{\beta_N}$ for all experiments reported in this paper. Legend is in the format {dataset}_l (model). Language refers to Fineweb-edu. $\overline{\alpha_D}$ and $\overline{\beta_N}$ are averages of the best fit exponents α_D and β_N over D and N . Error bars indicate standard deviation of the best fits α_D , β_N across different N and D respectively. **Right:** $\overline{\alpha_D}$ and $\overline{\beta_N}$ for 2-layer transformer experiments on language, T1L, T2L, and language bigrams, demonstrating monotonic evolution of $\overline{\alpha_D}$ with approximate entropy of the dataset, along with relative stability of $\overline{\beta_N} \approx 0.5$.

validate the power law fit, we compare it to an exponential fit of the form $f(x) = a + be^{-cx}$. The ratio between the mean squared error (MSE) of the power law and exponential fits are presented in Appendix C (Fig. 6). In all cases studied in this paper, the best power law fits for $L(D)_N$ have factors of 50-100x better MSEs than the best exponential fits; for $L(N)_D$, with one exception the power law fits have factors of 5-100x better MSEs than the exponential fits. An important message of our paper is that next-token prediction on various types of random walks gives power law fits to the loss that are on par with those for natural language.

Compute optimal fits require more analysis, because in practice we have not sampled enough values of N , D . In the next step, we obtain a prediction $\hat{L}(N, D)$ for values of N, D that are not in the raw dataset. To obtain this prediction, instead of using simple 2d formulas such as Eq. 2 as is common, we use an alternative technique – regression with a 3-layer fully connected neural network (for details see Appendix C). We demonstrate that this method (along with a kernel regression method) gives significantly better train/validation errors than the 2d Chinchilla fit that is widely used in the literature (see Figs. 5, Figs. 8, 9 in

can change significantly depending on whether N includes embedding parameters. Compare Fig. 20 with 21 (Appendix F).

Appendix F). Since the 2d Chinchilla fit does not have a strong theoretical basis, we suggest eschewing it in favor of more standard ML regression techniques. Given $\hat{L}(N, D)$, we can then use a sufficiently fine grid for N, D to extract compute optimal scaling laws following Eqs. 3, 4, 6.

In figure captions we report the mean exponents $\overline{\alpha_D}$ and $\overline{\beta_N}$, which are the average of the best-fit exponents α_D and β_N across different values of D and N . We also report the standard deviation for the best fit α_D and β_N across different values of D and N , which are the error bars in Fig. 2. Note that these standard deviations do not take into account the confidence intervals for each fit, which are shown in brackets in the figure legends. These averages and standard deviations are only summary statistics and do not contain the full information of α_D , β_N . In some cases we studied, namely those related to language data, α_D decreases monotonically with D , as displayed in Appendix D.

Details of our experiments are presented in Appendix B. We use a decoder-only GPT-style transformer to perform next-token prediction. We use settings characteristic of standard language pretraining (Brown et al., 2020; Hoffmann et al., 2022), including 1 epoch training with a linear warmup, cosine decay learning rate schedule. In our figures, “Compute” refers to the number of FLOPs (float-

ing point operations), “ N (embedding / non-embedding)” refers to the total number of parameters including / excluding embedding parameters.

Standard vs. maximal update parameterization (SP vs. μP). We use two different parameterizations, SP and μP , throughout the paper. The differences are in how the initialization variance, layer-wise learning rates, and parameter multipliers are scaled with the embedding dimension of the network, and are summarized in Appendix B.

3. Random walks, bigrams, and random graphs

A simple generative model of sequences is a Markov random walk on a graph. Consider a directed, weighted graph on V nodes with weight matrix W_{uv} . We can sample a random walk by randomly starting at a node with some initial distribution and then iteratively sampling from the probability distribution $p(u|v) := W_{uv} / \sum_u W_{uv}$. This is equivalent to generating from a bigram model. Recall that an n -gram language model approximates the conditional joint distribution of text in terms of the $n - 1$ most recent tokens: $p(x_t | x_{t-1}, \dots, x_0) = p_n(x_t | x_{t-1}, \dots, x_{t-n+1})$. In the case of a bigram model ($n = 2$), this defines a directed, weighted graph, which we refer to as the bigram graph. For a vocabulary of size V , the bigram graph has V nodes, and each directed edge uv is associated with a weight $p_2(u, v)$. Sampling from a bigram model corresponds to sampling a random walk on the bigram graph, with transition probabilities $p(u|v) = p_2(u, v) / p_1(v)$, where $p_1(v) = \sum_u p_2(u, v)$ is the stationary distribution on nodes (equivalently the unigram distribution of the sequence). This picture can be generalized to $n > 2$, where the graph is replaced with a hypergraph.

Next-token prediction on a random walk on an ordinary graph ($n = 2$) is essentially a problem of learning a probability table. Given training sequences $\{x_1^{(i)}, \dots, x_T^{(i)}\}$, for $i = 1, \dots, N_{\text{seq}}$, the task requires the model to learn that they are controlled by bigram transition probabilities $p(u|v)$ and then to learn the probabilities themselves. The obvious algorithm to learn this is to form a prediction $\hat{p}$ by counting co-occurrences in the training set, which is how bigram models are learned (Jurafsky & Martin, 2009). If we consider a cross-entropy or MSE loss function L comparing the prediction $\hat{p}$ to the ground truth p , then in expectation over a sample of size D tokens, $\mathbb{E}[L] = E + B/D + O(1/D^2)$, as shown in Appendix E. For cross-entropy loss, E is the entropy of the distribution p ; for MSE loss $E = 0$. B is a constant depending on $|V|$. This provides a baseline against which we can compare our empirical results.

An unbiased random walk on an undirected graph has the transition probability $p(u|v) = \frac{A_{uv}}{\deg(v)}$, where A is the ad-

jacency matrix and $\deg(v) = \sum_u A_{uv}$ is the degree of v . The stationary distribution is $p(u) = \frac{\deg(u)}{2E}$, where E is the number of edges in the graph. Recall that random walks have an exponentially decaying auto-correlation, with the time-scale for the exponential decay set by the spectral gap of the transition matrix.

To understand better the scaling laws of language models, we compare their performance to a simple baseline of learning random walks on random graphs.

3.0.1. ERDÖS-RENYI GRAPHS

Let V be the number of nodes. For the Erdős-Rényi ensemble, the adjacency matrix of the graph is chosen randomly such that $A_{ij} = 1$ with probability p and 0 with probability $1 - p$. The degree distribution of the nodes is tightly concentrated around the expected degree, $\langle k \rangle = p(V - 1)$ and the expected number of edges is $\langle E \rangle = \langle k \rangle V / 2 = pV(V - 1) / 2$. The spectrum of the adjacency matrix obeys a semicircle law, which is typical of random symmetric matrices. Therefore, neither the degree distribution nor the spectrum of the graph has any power laws. In Fig. 3 we present the degree distribution (equivalent to the unigram distribution for unbiased walks) and the bigram distribution ($p(u|v)$) for an unbiased random walk on an ER graph with $V = 8,192$ and $E = 53,292$ that we use, demonstrating the absence of any power law structure in the data correlations.

Our first main result is presented in Fig. 1, demonstrating the existence of neural scaling law behavior. This result demonstrates that neural scaling laws are a generic feature of learning sequence models with cross-entropy loss, *independent of whether the data itself has any power law structure*. These models were trained with maximal update parameterization (μP) with context length of 50; additional experimental details are presented in Appendix B.

We have studied unbiased random walks on ER graphs of varying sizes. Fig. 15 (Appendix F) demonstrates results from an ER graph with $V = 50,000$ and $E = 2\text{M}$ edges. We also present results for ER graphs with 1K nodes and 5K edges in Fig 13 (Appendix F); this demonstrates that scaling laws in these settings can be observed at relatively small scales, which should be accessible at relatively low compute levels ($10^{13} - 10^{16}$ FLOPs in our experiments).

3.0.2. BIASED RANDOM WALKS

We can controllably tune the stationary distribution (equivalently, unigram distribution) $p(v)$ and the transition probabilities $p(u|v)$ (equivalently, bigram distribution) by considering biased random walks. We consider sampling a weight matrix W_{uv} from a power-law distribution $\Pr(W_{uv} = k) \propto k^{-\kappa}$, for $k \in [k_{\min}, k_{\max}]$ and 0 other-

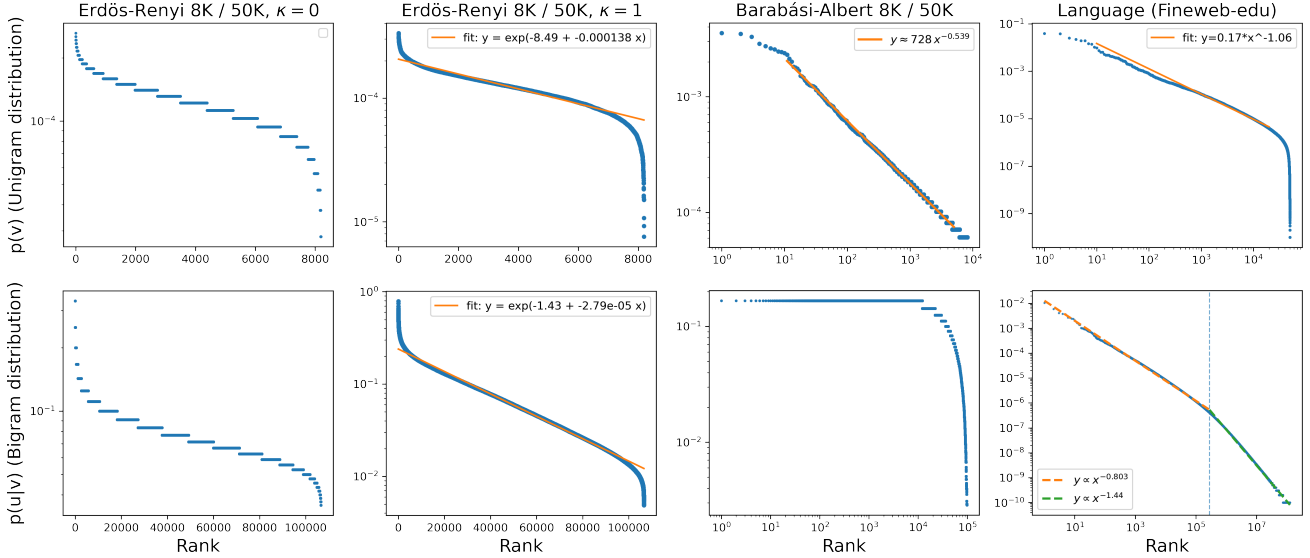


Figure 3. Stationary distribution on nodes $p(v)$ (unigram distribution) and transition probability distributions $p(u|v)$ (bigram distribution) plotted against the rank for various representative cases studied in this paper. Note that $p(v)$ is equal (up to an overall constant) to the degree distribution of the graph when $\kappa = 0$. Left two columns are plotted on log-linear scale, and the right two columns are on log-log scale. ER $\kappa = 0$ case is tightly concentrated around its mean probability, as indicated by roughly constant size plateaus. BA ($\kappa = 0$) has power laws in both unigram and bigram distributions; the unigram exponent is approximately equal to the theoretical expectation $1/(\gamma - 1) = 0.5$. The power law in the bigram is more clear if plotted in reverse rank order (not shown), but can be observed from the decreasing width of the plateaus. Language results are from Finweb-edu using GPT-2 tokenizer; unigram distribution is a standard Zipf plot, while bigram distribution shows broken power law.

wise (when the edge uv is present in the graph). For $\kappa > 1$, the transition probabilities follow a power law distribution, $P(u|v) \propto (1/r_{(u|v)})^{\frac{1}{\kappa-1}}$, where $r_{(u|v)}$ is the rank of the bigram $(u|v)$, sorted from highest to lowest in probability. $\kappa = 1$ is the marginal case, where the probabilities and their ranks follow an exponential (log-linear) relationship: $P(u|v) = P_0 e^{-sT(u|v)}$, for constants s, P_0 .

We present results for the marginal case $\kappa = 1$. The unigram and bigram distributions for the ER graph with 8K nodes and 50K edges are shown in Fig 3, which shows that $\kappa = 1$ smooths the distributions compared to the case $\kappa = 0$. This is an interesting intermediate case: on the one hand the data correlations still lack power laws, on the other hand they sit in between the $\kappa = 0$ case and the case where the unigram and bigram distributions have power laws.

The scaling law results for the marginal case $\kappa = 1$ are shown in Fig. 14 (Appendix F) for the 8K / 50K graph and in Fig. 16 (Appendix F) for the 50K / 2M graph. The primary observation is that the quality of the power law fits are improved (see, e.g. Fig. 6), suggesting that the structure of the data correlations can help crystallize power law scaling in the loss. It would be interesting to continuously tune the exponent κ and see the evolution in the scaling exponents and the quality of the fit, which we leave for future work.

3.0.3. SCALE-FREE GRAPHS: BARABÁSI-ALBERT

Another way to introduce power law structures in the data is to introduce it in the connectivity structure of the graph itself. For a scale-free graph (Barabási & Albert, 1999), a randomly chosen node has degree k with probability $\propto k^{-\gamma}$ for large k , with $\gamma > 1$. Co-occurrence matrices from natural language corpora have been shown to determine a scale-free, small-world graph (Cancho & Solé, 2001). One way to construct a scale-free graph is via the Barabási-Albert preferential attachment model (Barabási & Albert, 1999), which gives $\gamma = 3$. We use this model to construct a scale-free graph with 8,192 nodes and 49,131 edges. We display the power-law unigram and bigram distributions for the unbiased ($\kappa = 0$) walk in Fig. 3. The resulting scaling laws for learning unbiased random walks on this graph are presented in Fig. 12 (Appendix F). Qualitatively the results are similar to the ER graphs without power law structures.

4. Dialing up complexity

In order to better understand the origin of scaling laws, here we systematically tune the complexity of the dataset and track the changes in the scaling law exponents. At the lowest level of complexity we start with a bigram language model. At an intermediate level of complexity, we consider a synthetic dataset generated from sampling the outputs of

an n -layer transformer trained on FineWeb-edu (Penedo et al., 2024). For a sufficiently large transformer, given the success of large language models, these sequences should approximate natural language closely. For small transformers they presumably provide a synthetic dataset of intermediate complexity. In this paper we study the case $n = 1, 2, 4$, and refer to these datasets as Transformer-generated n -layer (TnL) datasets.

As a rough (albeit imperfect) measure of the complexity, we use the (inverse) entropy of the joint distribution of the sequences in the dataset. We think of this as a rough measure of complexity because a low complexity dataset has low signal in predicting x_t from $x_{t-1}, x_{t-2}, \dots$, giving rise to many likely possibilities and therefore high entropy. A high complexity dataset has high signal in predicting x_t from $x_{t-1}, x_{t-2}, \dots$, and therefore the entropy should be lowered. Indeed, it is easy to show that for an n -gram language model, the entropy monotonically decreases with n .

While one can define and compute the entropy of an n -gram model, it is not feasible to compute the entropy of the sequences generated by TnL since it would require a number of samples that is exponential in the sequence length. Even worse, the entropy of natural language does not have an intrinsic definition, since there is no true generative model that one can use to define a joint distribution. We can approximate the entropies by recalling that the cross-entropy loss is lower-bounded by the entropy of the reference distribution. Therefore, if we fit power laws as in Eq. 1, the entropy of the reference distribution can be approximated as $E = \lim_{N \rightarrow \infty} E_N = \lim_{D \rightarrow \infty} E_D$. As a rough estimate, we take the smallest value of E_N or E_D in our fits and use that as a proxy for E . This gives $E = 5.40, 4.04, 3.81, 3.18$ for the bigram language model, T1L, T2L, and natural language (Fineweb-edu), respectively, consistent with our expectation that these generative models have increasingly higher complexity.

Scaling laws for learning language bigrams. We measure scaling laws for next-token prediction on sequences generated from a bigram language model. To define our bigram language model, we use the GPT-2 tokenizer (Radford et al., 2019), with vocabulary size 50,257. We train our bigram model on the Fineweb-edu-10B dataset (Penedo et al., 2024). This dataset has approximately 130M unique bigrams. Fig. 3 (bottom right) displays the bigram probabilities $p(u, v)$ vs. their rank, showing a (broken) power-law distribution. As depicted in Fig. 11 (Appendix F), a large number of bigrams appear only a handful of times; for our work to be computationally tractable, we remove all bigrams that appear 5 times or fewer, leaving around 33M unique bigrams. Therefore, our truncated bigram model corresponds to a graph with approximately 50K nodes and 33M edges, and with a (broken) power law weight matrix.

The scaling laws for learning using a 2-layer transformer with maximal update parameterization (μP), a context length of 50, and a fixed batch size of 100 sequences are shown in Fig. 17 (Appendix F). The exponents α_D lie in the range $0.84 - 1.2$. The mean and standard deviation of the best fit exponents across the different values of D are $\overline{\alpha_D} = 0.977$, $\sigma_{\alpha_D} = 0.102$ respectively. The β_N lie in the range $0.47 - 0.60$, with mean $\overline{\beta_N} = 0.547$ and standard deviation $\sigma_{\beta_N} = 0.052$. While the α_N 's are significantly higher than natural language, the β 's are much closer. Interestingly, this language bigram example shows the clearest power law fits. As depicted in Fig. 6, the ratio of the MSE between the best power-law and best exponential fits are 0.0179 and 0.0026, respectively.

The compute optimal curves demonstrate that learning the bigram model is much more compute efficient than natural language, as the exponent γ is substantially larger, and also more parameter efficient, as a is almost half as large as b .

Scaling laws for learning TnL-Language. Our scaling law experiments for learning TnL sequences had the same settings as the language bigram experiments: 2-layer transformers, μP parameterization with a context length of 50 and a fixed batch size of 100 sequences. Fig. 23 and 24 (Appendix F) show the case with $n = 1$ and 2 respectively. The results for T4L were nearly identical to those for T2L, so we omit them from the presentation. A 2-layer transformer may not be able to clearly distinguish between sequences generated from a 2-layer and 4-layer transformer. As summarized in Fig. 2, we found that the mean of exponents $\overline{\alpha_D}$ interpolate between the language bigram and natural language results.

We caution that the trend in Fig. 2 is empirical and based on a relatively small number of datapoints. It would be interesting to perform additional detailed studies to understand better the correlation between $\overline{\alpha_D}$ and fitted entropy of the dataset and how this relates to other possible measures of complexity of the data.

Systematic dependence of α_D . We find that in all cases involving language bigrams, TnL, or natural language datasets, the exponent α_D monotonically decreases with D in a systematic way (see Appendix D, Fig. 10). The other random graph exponents do not demonstrate this behavior, the origin of which would be interesting to investigate further.

5. Revisiting scaling laws for natural language

In this section, we present some important observations regarding scaling laws for natural language modeling.

Importance of the irreducible loss. Our scaling laws did not show up as a line on a log-log plot. This is because of

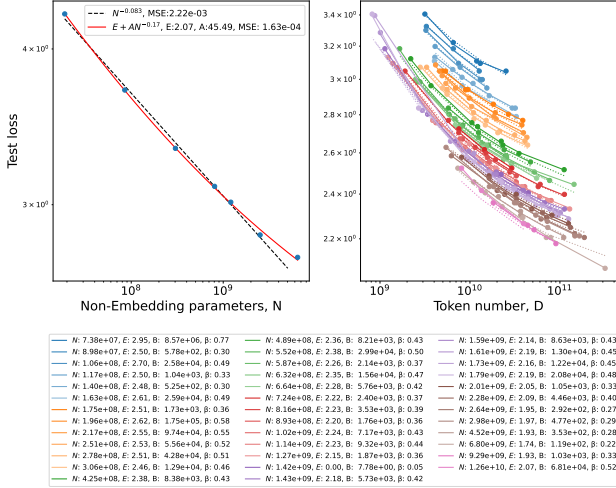


Figure 4. Left: Fits for $L(N)_D$ for the Pythia models. Right: 1d $L(D)_N$ fits for Chinchilla data (solid lines) with parameters shown in the legend, $L(N, D)$ fits to 2d Chinchilla formula (dotted lines). N includes embedding parameters.

the irreducible loss; we emphasize that dropping the constants E (referred to as the irreducible loss) in the 1d fits, as done in some prior works, leads to dramatic underestimation of the scaling exponents and significantly poorer fits. We consider six models in the Pythia sequence of LLMs (Biderman et al., 2023), which were trained on 300B tokens from the Pile dataset. Using the data provided by (Michaud et al., 2024), we consider the 1d fit $L(N)_D$ both with and without the offset E_D . Neglecting the offset E_D as in (Michaud et al., 2024) gives a nearly factor of two difference in the scaling exponent and almost an order of magnitude difference in the mean squared error (MSE) of the fit. We note that an exponent of $\alpha = 0.07$ was presented in (Henighan et al., 2020) (see top left of Fig. 3 in (Henighan et al., 2020)) upon neglecting E_D , and we expect a similar conclusion to hold in that case as well.

2d Chinchilla fit is relatively poor. The 2d Chinchilla formula in general gives poor fits compared to other standard ML methods. We suggest replacing the 2d Chinchilla formula with neural network regression to fit $L(N, D)$, which is the methodology used in this paper to obtain compute-optimal scaling results. We revisit the Chinchilla results (Hoffmann et al., 2022); the raw data for $L(N, D)$ reported in (Hoffmann et al., 2022) was extracted and shared in (Besiroglu et al., 2024), which we used for our analysis below. First, we compare three different methods for fitting a two-dimensional function $L(N, D)$, which allows determination of the compute optimal scaling laws: (i) fitting to the 2d Chinchilla formula, Eq. 2, (ii) neural network regression, and (iii) kernel regression. We refer to Appendix C for details regarding the neural network and kernel fits. The raw Chinchilla data contains 245 pairs of (N, D) val-

ues. Following (Besiroglu et al., 2024), we throw out the 5 largest outlier values of $L(N, D)$.

In all three cases, we perform an 80/20 train/validation split on the data, fit using a Huber loss ($\delta = 10^{-3}$) as in (Hoffmann et al., 2022), and report the MSE test loss averaged over 20 random splits. Such classical machine learning methods are not typically reported in the contemporary scaling law literature. The train / validation MSEs, averaged over the 20 train / validation splits, are displayed in Fig. 5. The neural network and kernel fits gives an almost $2\times$ factor improvement in the average validation MSE. From a model selection perspective, one should use the kernel or neural network fit over the 2d Chinchilla fit.

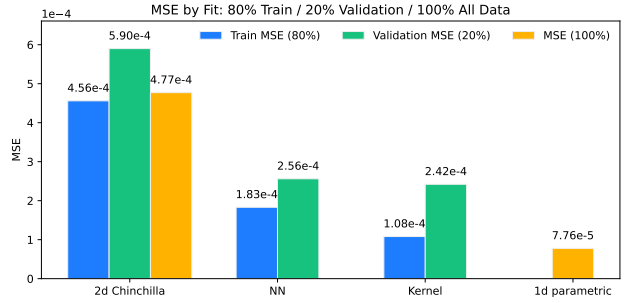


Figure 5. Train / validation MSEs for the 2d Chinchilla fit, neural network and kernel regressions, and the 1d parametric fit of Eq. 1, for Chinchilla data (Hoffmann et al., 2022; Besiroglu et al., 2024).

The drawback of the neural network and kernel fits are that they are not interpretable. To also have interpretable conclusions, we consider the 1d fit $L(D)_N$ given in Eq. 1. The result is plotted in Fig. 4 and the MSE is shown in Fig. 5. The 1d fits are significantly better, with a MSE of 7.76×10^{-5} . This is almost an order of magnitude better than the 2d fit, which has MSE of 4.77×10^{-4} . This is unsurprising as there are more parameters, but also indicates that it is more accurate to consider β_N varying with N .

The 2d fit yields the parameters $A = 477.82$, $B = 2143.62$, $E = 1.8172$, $\alpha = 0.3473$, $\beta = 0.3672$. The 1d fit, $L(D)_N = E_N + BD^{-\beta_N}$ gives a range of exponents, $\beta_N \sim 0.3 - 0.6$, albeit with some outliers; specifically, mean $\overline{\beta_N} = 0.41$ and standard deviation 0.085 after throwing out the two extreme outliers. While these are consistent with each other, β_N has considerable variation with N . There are not enough datapoints to fit $L(N)_D$.

One can also use the neural network or kernel fits to extract compute optimal curves, using the methodology described in Sec. 2.1. In Appendix C.4 Fig. 7, we present the results for compute optimal curves computed using the neural network fit to $L(N, D)$. Our method yields the exponents $\gamma = 0.16$, $a = 0.482$, $b = 0.504$. These are in relatively good agreement with the 2d Chinchilla fit ($\gamma = \frac{\alpha\beta}{\alpha+\beta} = 0.178$, $a = \frac{\beta}{\alpha+\beta} = 0.513$, $b = \frac{\alpha}{\alpha+\beta} = 0.487$).

Recovering basic scaling law results with 2 layer transformers and short context lengths. We revisit natural language modeling in the context of our experiments, to validate our methodology. We train a 2 layer transformer with context length 100, using a fixed batch size of 512 sequences, with standard parameterization (SP). The 1d fits for $L(N)_D$ and $L(D)_N$, where N either includes embedding parameters or not, are shown in Figs. 20, 21 (Appendix F). The α_D are in the range $\alpha_D \sim 0.29 - 0.46$, with a mean of $\overline{\alpha_D} = 0.354$, close to the result for α from the 2d Chinchilla fit. The β_N are in the range $0.48 - 0.55$, with a mean $\overline{\beta_N} = 0.511$. These are slightly higher, but within the same range, as the results in the preceding section where we refit the Chinchilla data. The 2d Chinchilla fit yields parameters $A \approx 507$, $B = 20094$, $E = 2.802$, $\alpha = 0.365$, $\beta = 0.503$. α, β are close to the means $\overline{\alpha_D}, \overline{\beta_N}$ obtained from 1d fits. These imply the compute optimal exponents $L_{\text{opt}} \sim C^\gamma$ with $\gamma = \frac{\alpha\beta}{\alpha+\beta} = 0.212$. In Fig. 20 (Appendix F), we present the compute optimal curves obtained using the neural network fit, which is within 20% of the prediction from the 2d Chinchilla formula.

Second, we examine compute optimal scaling in Figs. 20, 21 (Appendix F). When embedding parameters are included, the compute optimal exponents obey $a/b \approx 0.94$, which is close to the Chinchilla result $a/b \approx 1$. When embedding parameters are not included, we have $(a, b) = (0.742, 0.386)$, which is close to the OpenAI result of $(0.73, 0.27)$ (Kaplan et al., 2020). Thus, the discrepancy between Chinchilla (Hoffmann et al., 2022) and OpenAI (Kaplan et al., 2020) scaling laws can already be observed from training shallow transformers with short context lengths and arises from whether embedding parameters are included. This is in line with observations in (Porian et al., 2024; Pearce & Song, 2024). These prior works also identified learning rate schedules and other factors in contributing to the discrepancy; in light of our results, these might be more relevant for deeper models.

Scaling laws with μP : improved parameter efficiency? We examine 2, 4 layer transformers trained on Fineweb-edu, with μP parameterization, context length 50, and fixed batch size of 100 sequences. For both 2 and 4 layer transformers, we find $\overline{\alpha_D} \approx 0.5$, which is larger than the case of SP, which had $\overline{\alpha_D} \approx 0.35$. This indicates that these settings are more parameter efficient, as the loss decreases more sharply with N . This is also reflected in the compute optimal exponents $(a, b) \approx (0.42, 0.58)$, which shows that the optimal parameter size need not grow as quickly with compute as in the case of SP; *that is, compute optimal training is not at fixed token number per parameter*. Since μP gives $O(1)$ feature updates in the large width limit (Yang & Hu, 2021) it is plausible that it could be more parameter efficient for compute optimal training. It would be interesting to see if these results hold at larger scales.

6. Discussion

We demonstrated that scaling laws can be observed in the simpler setting of Markov random walks on graphs. We studied a wide variety of settings: (i) Erdős-Renyi graphs (1K nodes, 5K edges; 8K nodes, 50K edges; 50K nodes, 2M edges) with unbiased ($\kappa = 0$) and biased ($\kappa = 1$) random walks; (ii) scale free Barabási-Albert graphs (8K nodes, 50K edges); (iii) language bigram graphs (50K nodes, 33M edges). We also demonstrated that one can consider datasets of monotonically increasing complexity, from language bigrams, TnL, to natural language data, and that one can track the mean scaling exponents as the complexity is increased, revealing a monotonic decrease in $\overline{\alpha_D}$.

Upon analyzing the scaling exponents, we see that most of them appear to the right of the $\overline{\alpha_D} = \overline{\beta_N}$ line in Fig. 2, suggesting that the loss is limited more by data than by architectural capacity. This may be related to our setting of 1 epoch training, where the number of training steps is tied to the dataset size; relaxing this might make the models more sample efficient and increase $\overline{\beta_N}$. It is also likely related to the fact that the setting of Markov random walks is too simple to be limited by architectural capacity. It would be interesting to find settings that are more limited by architectural capacity than by dataset size, so that $\overline{\beta_N} \gg \overline{\alpha_D}$; this would be expected in more complex datasets, where larger models are required, but where data is used more effectively, for example through multi-epoch training, data augmentation, synthetic data, self-play, and so on.

One of our key points, as exhibited by our results on random walks on ER graphs (Fig. 1), is that empirically scaling laws arise even when the data itself has no power law correlations. This calls for deeper or alternative explanations than have been provided so far in the literature. In the setting of one epoch training for a fixed dataset size D , the number of optimization steps is also fixed. As we increase N with fixed D , the scaling laws tell us that the model is able to reach a lower loss with the same number of training steps. Equivalently, the model is able to reach a given value of the loss with fewer steps. This suggests that the basin of low loss solutions is opening up in a particular way with model size, which allows gradient-based optimization to be more efficient. A potential path forward could be to study how the loss landscape is opening up with N and how this affects the gradients as a function of N during training.

In the kernel regression models referenced earlier, power law scaling in the loss arose from power laws in the features. While we have examples where the data has no power laws, it might be the case that power laws develop in the internal activations of the model, which then drive power law scaling in the loss. It would be interesting to further study these internal activations and see to what extent signatures of power laws arise.

7. Acknowledgments

We thank Newsha Ardalani, Yasaman Bahri, and Andrew Cohen for discussions and Rylan Schaeffer, Dan Roberts, and Cengiz Pehlevan for comments on the draft. MB thanks the Simons Collaboration on Physics of Learning and Neural Computation (SFI-MPS-POL-00012574-09) for support while at UMD.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Altmann, E. G. and Gerlach, M. Statistical laws in linguistics. In *Creativity and universality in language*, pp. 7–26. Springer, 2016.
- Bahri, Y., Dyer, E., Lee, J., Kaplan, J., and Sharma, U. Explaining neural scaling laws. *arXiv preprint arXiv:2102.06701*, 2021. URL <https://arxiv.org/abs/2102.06701>.
- Barabási, A.-L. and Albert, R. Emergence of scaling in random networks. *science*, 286(5439):509–512, 1999.
- Besiroglu, T., Erdil, E., Barnett, M., and You, J. Chinchilla scaling: A replication attempt, 2024. URL <https://arxiv.org/abs/2404.10102>.
- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., and Hoefler, T. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, March 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i16.29720. URL <http://dx.doi.org/10.1609/aaai.v38i16.29720>.
- Bhagia, A., Liu, J., Wettig, A., Heineman, D., Tafjord, O., Jha, A. H., Soldaini, L., Smith, N. A., Groeneveld, D., Koh, P. W., Dodge, J., and Hajishirzi, H. Establishing task scaling laws via compute-efficient model ladders, 2025. URL <https://arxiv.org/abs/2412.04403>.
- Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O’Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.
- Black, S., Biderman, S., Hallahan, E., Anthony, Q., Gao, L., Golding, L., He, H., Leahy, C., McDonell, K., Phang, J., et al. Gpt-neox-20b: An open-source autoregressive language model. In *Proceedings of BigScience Episode# 5-Workshop on Challenges & Perspectives in Creating Large Language Models*, pp. 95–136, 2022.
- Bordelon, B., Canatar, A., and Pehlevan, C. Spectrum dependent learning curves in kernel regression and wide neural networks. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 1024–1034. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/bordelon20a.html>.
- Bordelon, B., Atanasov, A., and Pehlevan, C. A dynamical model of neural scaling laws. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 4345–4382. PMLR, 2024. URL <https://proceedings.mlr.press/v235/bordelon24a.html>.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Cagnetta, F., Petrini, L., Tomasini, U. M., Favero, A., and Wyart, M. How deep neural networks learn compositional data: The random hierarchy model. *Physical Review X*, 14(3):031001, 2024.
- Cancho, R. F. I. and Solé, R. V. The small world of human language. *Proceedings of the Royal Society of London. Series B: Biological Sciences*, 268(1482):2261–2265, 2001.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., and et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL <https://arxiv.org/abs/2204.02311>.
- Clauset, A., Shalizi, C. R., and Newman, M. E. J. Power-law distributions in empirical data. *SIAM Review*, 51(4):661–703, 2009. doi: 10.1137/070710111. URL <https://doi.org/10.1137/070710111>.
- Cortes, C., Jackel, L. D., Solla, S., Vapnik, V., and Denker, J. Learning curves: Asymptotic values and rate of convergence. *Advances in neural information processing systems*, 6, 1993.
- Dey, N., Gosal, G., Khachane, H., Marshall, W., Pathria, R., Tom, M., Hestness, J., et al. Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster. *arXiv preprint arXiv:2304.03208*, 2023.
- Edelman, E., Tsilivis, N., Edelman, B., Malach, E., and Goel, S. The evolution of statistical induction heads: In-context learning markov chains. *Advances in neural information processing systems*, 37:64273–64311, 2024.
- Gadre, S. Y., Smyrnis, G., Shankar, V., Gururangan, S., Wortsman, M., Shao, R., Mercat, J., Fang, A., Li, J., Keh, S., Xin, R., Nezhurina, M., Vasiljevic, I., Jitsev, J., Soldaini, L., Dimakis, A. G., Ilharco, G., Koh, P. W., Song, S., Kollar, T., Carmon, Y., Dave, A., Heckel, R., Muennighoff, N., and Schmidt, L. Language models scale reliably with over-training and on downstream

- tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2403.08540>.
- Grattafiori, A. and et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Henighan, T., Kaplan, J., Katz, M., Chen, M., Hesse, C., Jackson, J., Jun, H., Brown, T. B., Dhariwal, P., Gray, S., et al. Scaling laws for autoregressive generative modeling. *arXiv preprint arXiv:2010.14701*, 2020.
- Hestness, J., Narang, S., Ardalani, N., Diamos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., and Zhou, Y. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Hutter, M. Learning curve theory, 2021. URL <https://arxiv.org/abs/2102.04074>.
- Jiang, A. Q. and et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL <https://arxiv.org/abs/2401.04088>.
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 2009. ISBN 978-0131873216.
- Kalra, D. S. nanogpt-mup, 2025. URL <https://github.com/dayal-kalra/nanoGPT-mup>. commit 1e9b690bf91377f3cb2d4c9b4033d849c3303f5d.
- Kang, F., Ardalani, N., Kuchnik, M., Emad, Y., Elhoushi, M., Sengupta, S., Li, S.-W., Raghavendra, R., Jia, R., and Wu, C.-J. Demystifying synthetic data in llm pre-training: A systematic study of scaling laws, benefits, and pitfalls, 2025. URL <https://arxiv.org/abs/2510.01631>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Khona, M., Okawa, M., Hula, J., Ramesh, R., Nishi, K., Dick, R., Lubana, E. S., and Tanaka, H. Towards an understanding of stepwise inference in transformers: A synthetic graph navigation model, 2024. URL <https://arxiv.org/abs/2402.07757>.
- Li, M., Kudugunta, S., and Zettlemoyer, L. (mis) fitting scaling laws: A survey of scaling law fitting techniques in deep learning. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Lin, L., Wu, J., Kakade, S. M., Bartlett, P. L., and Lee, J. D. Scaling laws in linear regression: Compute, parameters, and data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://neurips.cc/virtual/2024/poster/95321>.
- Liu, A. and et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024. URL <https://arxiv.org/abs/2405.04434>.
- Liu, Y., Liu, Z., and Gore, J. Superposition yields robust neural scaling. *arXiv preprint arXiv:2505.10465*, 2025.
- Makkuva, A. V., Bondaschi, M., Girish, A., Nagle, A., Jaggi, M., Kim, H., and Gastpar, M. Attention with markov: A curious case of single-layer transformers. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Maloney, A., Roberts, D. A., and Sully, J. A solvable model of neural scaling laws. *arXiv preprint arXiv:2210.16859*, 2022.
- Michaud, E. J., Liu, Z., Girit, U., and Tegmark, M. The quantization model of neural scaling, 2024. URL <https://arxiv.org/abs/2303.13506>.
- Muennighoff, N., Rush, A. M., Barak, B., Le Scao, T., Piktus, A., Tazi, N., Pyysalo, S., Wolf, T., and Raffel, C. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.16264>.
- Paquette, E., Paquette, C., Xiao, L., and Pennington, J. 4+3 phases of compute-optimal neural scaling laws. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/1dcccfc3ee01871d05e33457c61037d59-Abstract-Conference.html.
- Pearce, T. and Song, J. Reconciling kaplan and chinchilla scaling laws. *arXiv preprint arXiv:2406.12907*, 2024.
- Penedo, G., Kydlíček, H., Lozhkov, A., Mitchell, M., Raffel, C. A., Von Werra, L., Wolf, T., et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

- Perozzi, B., Al-Rfou, R., and Skiena, S. Deepwalk: on-line learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD '14, pp. 701–710. ACM, August 2014. doi: 10.1145/2623330.2623732. URL <http://dx.doi.org/10.1145/2623330.2623732>.
- Piantadosi, S. T. Zipf’s word frequency law in natural language: A critical review and future directions. *Psychonomic bulletin & review*, 21(5):1112–1130, 2014.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *Advances in Neural Information Processing Systems*, 37:100535–100570, 2024.
- Press, O. and Wolf, L. Using the output embedding to improve language models. *EACL 2017*, pp. 157, 2017.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Ren, Y., Nichani, E., Wu, D., and Lee, J. D. Emergence and scaling laws in sgd learning of shallow neural networks. *arXiv preprint arXiv:2504.19983*, 2025.
- Ruderman, D. L. The statistics of natural images. *Network: computation in neural systems*, 5(4):517, 1994.
- Sharma, U. and Kaplan, J. Scaling laws from the data manifold dimension. *Journal of Machine Learning Research*, 23(9):1–34, 2022.
- Shi, W. and Cao, Y. Towards understanding transformers in learning random walks. *arXiv preprint arXiv:2511.23239*, 2025.
- Song, J., Liu, Z., Tegmark, M., and Gore, J. A resource model for neural scaling law, 2024. URL <https://arxiv.org/abs/2402.05164>.
- Spigler, S., Geiger, M., and Wyart, M. Asymptotic learning curves of kernel methods: empirical data versus teacher–student paradigm. *Journal of Statistical Mechanics: Theory and Experiment*, 2020(12):124001, December 2020. ISSN 1742-5468. doi: 10.1088/1742-5468/abc61d. URL <http://dx.doi.org/10.1088/1742-5468/abc61d>.
- Tian, C., Chen, K., Liu, J., Liu, Z., Zhang, Z., and Zhou, J. Towards greater leverage: Scaling laws for efficient mixture-of-experts language models. *arXiv preprint arXiv:2507.17702*, 2025. URL <https://arxiv.org/abs/2507.17702>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Yang, A. and et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024. URL <https://arxiv.org/abs/2407.10671>.
- Yang, A. and et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Yang, G. and Hu, E. J. Tensor programs iv: Feature learning in infinite-width neural networks. In *International Conference on Machine Learning*, pp. 11727–11737. PMLR, 2021.
- Zhang, G., Qu, S., Liu, J., Zhang, C., Lin, C., Yu, C. L., Pan, D., Cheng, E., Liu, J., Lin, Q., et al. Map-neo: Highly capable and transparent bilingual large language model series. *arXiv preprint arXiv:2405.19327*, 2024.

A. Additional related work

Transformers and random walks / Markov chains. (Perozzi et al., 2014; Shi & Cao, 2025; Makuva et al., 2025; Edelman et al., 2024) study the ability of neural networks to learn random walks or Markov processes, often focusing on model capability and learnability. Our work instead focuses on using random walks on graphs as a paradigm for understanding neural scaling laws.

Neural scaling laws Several works in addition to those referenced earlier also examine the origin of neural scaling laws. (Sharma & Kaplan, 2022; Bahri et al., 2021) show that the existence of a data manifold implies neural scaling laws. The data in our work consists of sequences of discrete tokens, which have no obvious data manifold, so such an explanation appears to be inapplicable.

Other mechanisms for neural scaling laws that have been studied include (Liu et al., 2025), which isolates “superposition” as an important underlying principle, and (Song et al., 2024) which proposes a resource-based theory. It would be interesting if these perspectives could be related to scaling laws we observed in the context of random walks. (Hutter, 2021) was an early paper suggesting that power laws in the data were responsible for “interesting” scaling laws (i.e. data exponents $\alpha \neq 0.5$ or 1) with respect to dataset size, although there is no neural network architecture and associated model parameter scaling law in Hutter’s model.

Scaling laws with μP (Dey et al., 2023) studied compute-optimal scaling in the case of fixed tokens-per-parameter, finding that μP gave $\sim 1\%$ better loss and more robust scaling laws. We are not aware of reports in the literature of the scaling law exponents for μP ; in this paper we find that μP is more parameter-efficient (larger $\overline{\alpha_D}$ and smaller a) than SP at relatively small scales.

B. Experimental details

Parametrization	Parameter	Init. Var.	Param. Mult.	Gradient	Adam LR
SP	Embedding	1	1	$1/\sqrt{n}$	1
	Hidden	$1/n$	1	$1/\sqrt{n}$	$1/n$
	Readout	$1/n$	1	1	$1/n$
μP	Embedding	$1/n$	$\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$
	Hidden	$1/n$	1	$1/n$	$1/n$
	Readout	$1/n$	$1/\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$

Table 1. Scaling rules for standard parametrization (SP) and μP . In the implementation of μP that we use, only matrix weights receive weight decay; biases and LayerNorm parameters do not; LayerNorm learning rates are $O(1)$ while attention logits are scaled by $1/\text{head_dim}$.

Transformer architectures. We train pre-normalization decoder-only transformer models (Vaswani et al., 2017; Radford et al., 2019) using rotary positional embedding (Black et al., 2022). We follow the rotary implementation available on Huggingface, with the default frequency base of 10,000 and with rotary dimension equal to the embedding dimension. The model vocabulary size is V , where V is the number of nodes in the graph. The models have 2 or 4 hidden layers, with embedding dimension ranging from $n_{\text{embd}} = 128$ to 4096. For each model, we set the number of attention heads to $\max(4, n_{\text{embd}}/64)$. No tokenizer is required, as each node in our graphs is treated as a distinct token with its own unique token ID. We tie the token embedding matrix and the output head weights (Press & Wolf, 2017).

Optimization. We initialize all models from scratch and pre-train them by minimizing the cross-entropy loss between the ground-truth data and predicted sequences. Model parameters are optimized using AdamW with a weight decay of 0.01 and a peak learning rate λ . We sweep λ over a grid of 14 values ranging from $5 \cdot 10^{-6}$ to 0.1. We use a learning rate schedule with a linear warmup for the first 2% of training steps, followed by cosine decay to 0. On each training step, we stack 100 independent random-walks into a tensor of shape $(100, T)$. We use the parameter initialization scheme and μP implementation from the nanoGPT-muP repository (Kalra, 2025); we found that μP helped mitigate training instabilities across the sweep of learning rates for our random walk experiments. The differences between μP and SP are summarized in Table 1, where n is the embedding dimension (width) of the network. For each hyperparameter configuration, we train 3 models with different random initialization seeds and we report the best test loss performance.

Data generation. We introduce synthetic random-walk datasets. We generate sequences using the following procedure:

- For a desired sequence length T (generally 50 or 100), we start from a weighted transition matrix W_{uv} and an initial-node distribution M_u where $\sum_v W_{uv} = 1$ for every node u and $\sum_u M_u = 1$. W_{uv} is the probability of moving from node u to node v in one step and M_u is the probability of choosing node u as the starting point.
- We sample the first node of each sequence from M_u .
- We then sample the remaining $T - 1$ nodes by repeatedly drawing the next node v according to W_{uv} , where u is the current node.
- These walks may revisit nodes and traverse the same edge multiple times (i.e. node and edge repetition is allowed).

For Erdős-Renyi or Barabási-Albert graphs, we first create a graph $\mathcal{G}$ with V nodes and E edges, yielding a binary symmetric adjacency matrix A_{uv} . For biased walks, we sample W_{uv}^{init} from a power-law distribution $\Pr(W_{uv}^{init} = k) \propto k^{-\kappa}$, for $k \in [k_{\min}, k_{\max}]$ whenever $A_{uv} = 1$ and set $W_{uv}^{init} = 0$ otherwise. For unbiased walks, we let $W_{uv}^{init} = A_{uv}$.

For bigram graphs, we first tokenize the Fineweb-edu-10B dataset (Penedo et al., 2024) using the GPT-2 tokenizer (Radford et al., 2019), which has a vocabulary of size 50,257. This tokenized dataset has approximately 130M unique bigrams. We remove all bigrams that appear 5 times or fewer, leaving around 33M unique bigrams. Let C_{uv} denote the number of occurrences of the bigram (u, v) in the filtered corpus and set $W_{uv}^{init} = C_{uv}$.

Finally, we define

$$W_{uv} = \frac{W_{uv}^{init}}{\sum_j W_{uj}^{init}} \quad \text{and} \quad M_u = \frac{\sum_j W_{uj}^{init}}{\sum_{i,j} W_{ij}^{init}}.$$

For T1L, T2L and T4L, we train a n -layer transformer (respectively $n = 1, 2, 4$) with an embedding dimension of 4096 and a vocabulary size of 50,257 on the Fineweb-edu-10B dataset (Penedo et al., 2024). We train with a context length of 50, batch size of 100 and choose the learning rate λ that minimizes the training loss. After training, we sample random sequences from the model using temperature 1.

All natural language experiments are done with Fineweb-edu.

C. Fitting scaling laws

C.1. $L(N)_D$ and $L(D)_N$ fits

To fit $L(D)_N$ and $L(N)_D$, we fit the power law

$$y(x) = E + B x^{-\beta} \quad (\beta > 0)$$

by minimizing a Huber-robust least-squares objective with cutoff δ . Specifically, for residuals $r_i = y(x_i; \theta) - y_i$, we minimize $\sum_i \rho_\delta(r_i)$, where

$$\rho_\delta(r) = \begin{cases} \frac{1}{2} r^2, & |r| \leq \delta, \\ \delta (|r| - \frac{1}{2} \delta), & |r| > \delta, \end{cases}$$

so errors smaller than δ are treated quadratically while larger errors are penalized approximately linearly (down-weighting outliers). We set $\delta = 1.4826 \text{MAD}(y)$, where $\text{MAD}(y) = \text{median}(|y - \text{median}(y)|)$ is the median absolute deviation, and provides an estimate for an appropriate scale of δ . (If y is pure Gaussian noise, this reduces to $\delta = \sigma$, the standard deviation of the Gaussian noise; on the other hand, if y is dominated by signal and noise is small, δ is large and the fit reduces to ordinary least squares). We use a fallback $\delta = 0.1 \text{std}(y)$ if the median absolute deviation is zero. For numerical stability we reparameterize as

$$y(x) = E + A \left(\frac{x}{x_0} \right)^{-\beta}, \quad x_0 = \text{median}(x), \quad A > 0,$$

optimize over $(E, \log A, \log \beta)$ with bounds $\beta \in [\beta_{\min}, \beta_{\max}]$, and then report $B = A x_0^\beta$. The bounded nonlinear optimization (trust-region reflective algorithm) is run from 40 initializations: E_0 is chosen from y -quantiles, with (A_0, β_0)

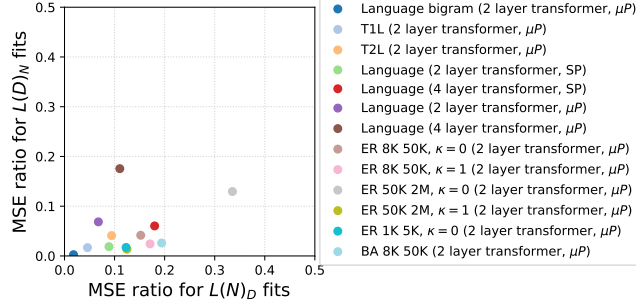


Figure 6. Ratio of the mean-squared error between the best power-law and the best exponential ($y = a + be^{-cx}$ fits, in order to validate the power-law scaling hypothesis. The power law fits for $L(D)_N$ are extremely good. The ones for $L(N)_D$ are generally not as clean, but it is unambiguous that the power law form gives a substantially better fit. Biased random walks with power-law weights give clearer power-laws than unbiased random walks for Erdős-Renyi graphs.

obtained from an approximate log-linear fit of $\log(y - E_0)$ versus $\log(x/x_0)$ on points where $y - E_0 > 0$, plus additional random starts; we keep the solution with the smallest final Huber objective.

For all of our $L(N)_D$ and $L(D)_N$ fits, we compare the quality of the power law fit to the quality of the best exponential fit, $y = a + be^{-cx}$, by examining the MSEs of the power law and exponential fits. The ratios of the power law and exponential MSEs for the datasets studied in this paper are shown in Fig. 6. This demonstrates that the power law fits for $L(D)_N$ are extremely good (a factor of 50-100x better than the best exponential fits). The power law fits for $L(N)_D$ are also good; with one exception, they are between 5-100x better than the best exponential fits.

C.2. Confidence intervals

In our fits we report 95% confidence intervals (shown in square brackets in the figure legends) for $\theta = (E, B, \beta)$ in the power law $y(x) = E + Bx^{-\beta}$ using a fixed- x wild bootstrap with BCa correction. After obtaining the robust point estimate $\hat{\theta}$ and fitted values $\hat{y}_i = \hat{E} + \hat{B}x_i^{-\hat{\beta}}$, we form residuals $r_i = y_i - \hat{y}_i$. For each bootstrap replicate $b = 1, \dots, N_{\text{boot}}$, we generate

$$y_i^{*(b)} = \hat{y}_i + \varepsilon_i^{(b)} r_i, \quad (7)$$

with i.i.d. weights $\varepsilon_i^{(b)} = \pm 1$ satisfying $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = 1$. We refit the model to $\{(x_i, y_i^{*(b)})\}$ using the same Huber objective and the same cutoff δ as in the base fit, warm-starting the optimizer from the base solution; failed refits are discarded.

For each scalar parameter $t \in \{E, B, \beta\}$ we obtain bootstrap draws $\{\hat{t}^{*(b)}\}_{b=1}^{N_{\text{boot}}}$ and use BCa (bias-corrected and accelerated) intervals. The *bias-correction* term is

$$z_0 = \Phi^{-1} \left(\frac{\#\{\hat{t}^{*(b)} < \hat{t}\} + \frac{1}{2} \#\{\hat{t}^{*(b)} = \hat{t}\}}{N_{\text{boot}}} \right), \quad (8)$$

which measures how the bootstrap distribution is shifted relative to the point estimate $\hat{t}$ (here Φ is the standard normal CDF). The *acceleration* term a is estimated via a leave-one-out jackknife: for each $i = 1, \dots, n$ we refit the model on the dataset with observation i removed, yielding jackknife estimates $t_{(i)}$ and mean $\bar{t}_{(\cdot)} = \frac{1}{n} \sum_i t_{(i)}$. We then set

$$a = \frac{\sum_{i=1}^n (\bar{t}_{(\cdot)} - t_{(i)})^3}{6 \left[\sum_{i=1}^n (\bar{t}_{(\cdot)} - t_{(i)})^2 \right]^{3/2}}. \quad (9)$$

Given nominal quantiles $\alpha/2$ and $1 - \alpha/2$ (with $\alpha = 0.05$ for 95% CIs), we compute adjusted quantile levels

$$\alpha_{\text{BCa}}(q) = \Phi \left(z_0 + \frac{z_0 + \Phi^{-1}(q)}{1 - a(z_0 + \Phi^{-1}(q))} \right), \quad (10)$$

and take the endpoints as the empirical $\alpha_{\text{BCa}}(\alpha/2)$ and $\alpha_{\text{BCa}}(1 - \alpha/2)$ quantiles of $\{\hat{t}^{*(b)}\}$. We also report bootstrap standard errors as the sample standard deviation of $\{\hat{t}^{*(b)}\}$.

For the $L(D)_N$ and $L(N)_D$, we use $N_{\text{boot}} = 4000$ bootstrap samples.

C.3. Fitting $L(N, D)$

We compare three different methods for fitting a two-dimensional function $L(N, D)$, which allows determination of compute optimal scaling laws. In all cases below, we use the Huber loss, with $\delta = 1 \times 10^{-3}$ to match the setting in (Hoffmann et al., 2022; Besiroglu et al., 2024).

Fitting with 2d Chinchilla formula. We fit to Eq. 2 following the methodology of (Hoffmann et al., 2022). We use the code and data from the github repository of (Besiroglu et al., 2024).

Neural network regression with 3-layer FCN. We fit $L(N, D)$ with a 3-layer fully connected neural network containing linear layers of dimension $2 \times n$, $n \times n$, and $n \times 1$ and GeLU activations. For fitting the Chinchilla data, we used a width $n = 512$; for the rest of the fits we use $n = 256$. The model achieving the best loss is used, within 5000 epochs of AdamW (weight decay $1e - 4$) or until the loss has stopped improving by more than $1e - 6$ for more than 200 steps.

Kernel ridge regression. We fit $L(N, D)$ with the ANOVA-style RBF kernel

$$k(x, x') = w_n \exp\left(-\frac{(n-n')^2}{2\ell_n^2}\right) + w_d \exp\left(-\frac{(d-d')^2}{2\ell_d^2}\right) + w_{nd} \exp\left(-\frac{(n-n')^2 + (d-d')^2}{2\ell_{nd}^2}\right),$$

with nonnegative amplitudes $w_{\{\cdot\}}$ and length-scales $\ell_{\{\cdot\}}$, where $x = (n, d) = (\log_{10} N, \log_{10} D)$. This kernel induces the function decomposition $f(x) = f_n(n) + f_d(d) + f_{nd}(n, d)$, recovering a standard RBF when $w_n = w_d = 0$. We set $w_{\{\cdot\}} = \ell_{\{\cdot\}} = 1$, although in principle a hyperparameter search can be carried out. The inputs x are centered and normalized and the outputs are centered. The regression minimizes the objective $\sum_i \rho_\delta(y_i - f_i) + (\lambda/2)\alpha^T K \alpha$, with $f = K\alpha$, where ρ_δ is the Huber loss. This is solved by iteratively reweighted least squares (IRLS) with per-sample weights $w_i = 1$ if $|r_i| \leq \delta$ and $w_i = \delta/|r_i|$ otherwise, and a linear system at each step: $(WK + \lambda I)\alpha = Wy_c$

C.4. Fitting compute optimal scaling laws, $L_{\text{opt}}, N_{\text{opt}}, D_{\text{opt}}$

We use the fits $\hat{L}(N, D)$ obtained using the neural network regression approach. We pick a grid of 50 or 100 points between the minimum and maximum values of N in our raw data, and similarly for D . We then determine the compute optimal loss in two ways, by $\min_N \hat{L}(N, C/6N)$ and $\min_D \hat{L}(C/6D, D)$. For a sufficiently dense grid and smooth enough fit $\hat{L}$, these two should coincide. This gives us an estimate of the compute optimal loss $L_{\text{opt}}(C)$. We fit $L_{\text{opt}}(C)$ to the power law form using the same methodology we used for $L(D)_N$ and $L(N)_D$, although we instead use 200 bootstrap samples for the confidence intervals since we observed the confidence intervals are already stable for those values of N_{boot} .

Next, from the fit $\hat{L}(N, D)$ over the grid described above, we obtain $N_{\text{opt}}(C)$ and $D_{\text{opt}}(C)$ using Eq. 4. We fit then do a standard linear regression to fit $\log N_{\text{opt}}(C)$ and $\log D_{\text{opt}}(C)$ to a line to extract the compute optimal exponents a and b .

In almost cases, we have to clip by hand the lower and upper values of C . One reason is that near the boundaries $C_{\min} = 6N_{\min}D_{\min}$ and $C_{\max} = 6N_{\max}D_{\max}$, the fit is less accurate and so $L_{\text{opt}}(C)$, N_{opt} , and D_{opt} exhibit spurious behavior in these limits. Another reason is that in some cases, N_{opt} and D_{opt} are beyond the grid for N, D we have sampled.

We observed in practice that the neural network regression usually appears to give more sensible results for the compute optimal curves as compared with the kernel regression approach, so in all of our figures we report results using the neural network regression. We eschew the 2d Chinchilla fit for the neural network regression approach because it gives significantly better MSEs for the fits, as shown in Figs. 5, 8, 9, and explained in the main text.

In nearly all cases, we found that the parameters of the fits for the scaling laws are sensitive to the window chosen for the fits. In principle the confidence intervals should also include a sensitivity analysis with respect to the window of the fit; we leave a detailed such analysis, which is of particularly important in for natural language data, for future work.

Fig. 7 demonstrates the neural network regression approach introduced above for obtaining compute optimal scaling curves for the raw Chinchilla data that we reanalyzed in Sec. 5. As described in the main text, the exponents are close to those obtained via other methods in (Hoffmann et al., 2022; Besiroglu et al., 2024).

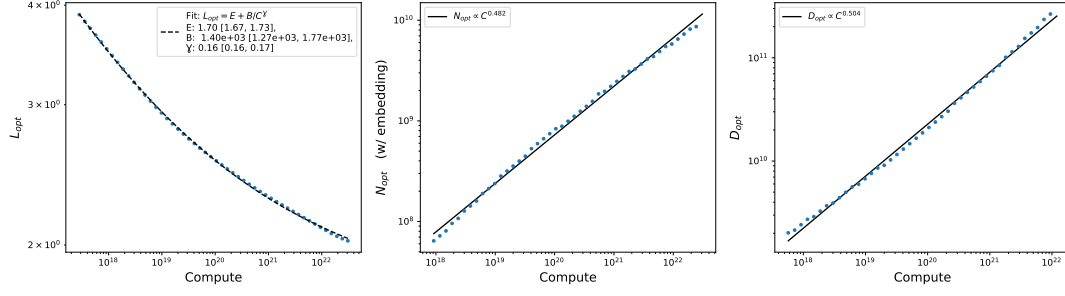


Figure 7. Compute optimal scaling laws obtained using the neural network fit for $L(N, D)$.

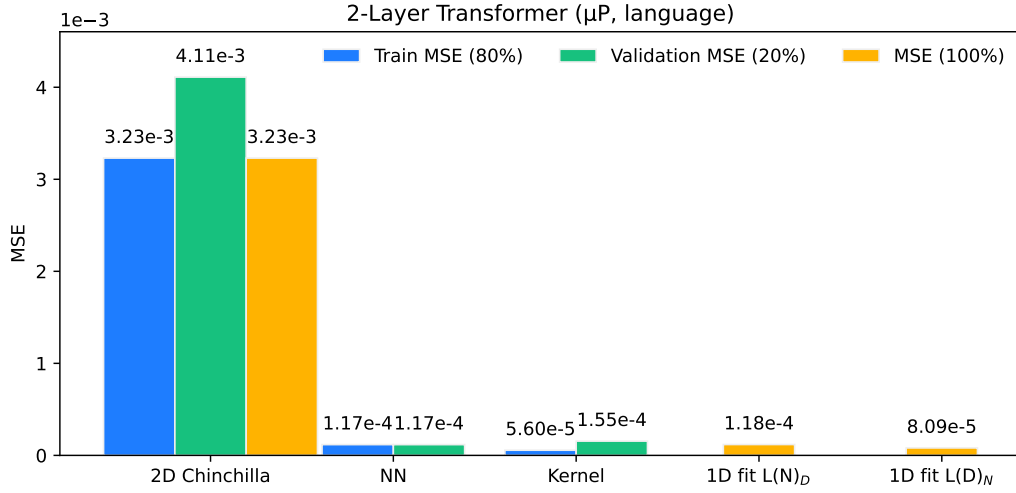


Figure 8. The 2d Chinchilla fit is around $30 - 40\times$ worse than the NN and kernel fits. N = embedding parameters. Non-embedding is similar.

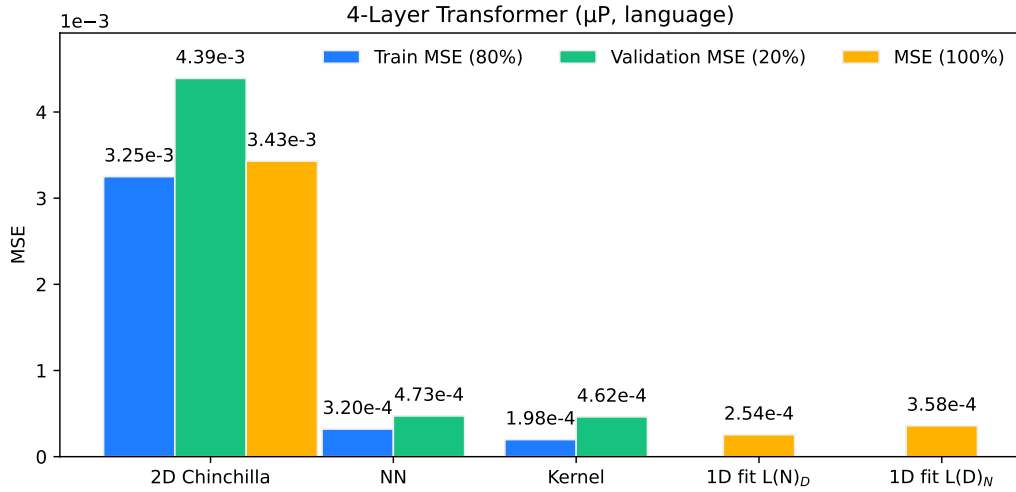


Figure 9. N = embedding parameters. Non-embedding is similar. 2d Chinchilla fit yields the parameters $A = 1838$, $B = 677$, $E = 3.342$, $\alpha = 0.484$, $\beta = 0.378$

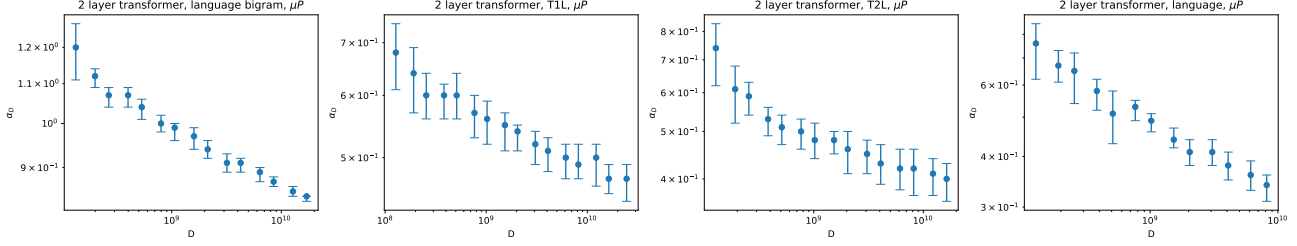


Figure 10. Systematic dependence of α_D on D for the various scaling law fits shown earlier, based on language bigrams, TnL, and natural language.

D. α_D vs. D

For the language-related datasets, we found that the fitted values of the exponent α_D exhibits a systematic dependence with D , shown in Fig. 10. The other language related cases studied in this paper, e.g. using SP, also show similar behavior. We do not observe similar systematic behavior in β_N or in the exponents coming from random graphs. It would be interesting to study this structure in more detail and understand its origin.

E. Sample error baseline

Consider the problem of estimating a probability distribution π over a finite sample space V . Given a dataset of D samples, consider the counting estimate

$$\hat{\pi}_a = \frac{n_a}{D}, \quad (11)$$

where n_a is the number of times $a \in V$ appears in the dataset. Here we derive the data scaling laws for this counting model for cross-entropy and MSE losses, which provides a baseline against which to compare the random walk scaling law results. Note that this model was also studied in the Appendix of (Hestness et al., 2017) using a loss based on L1 norm, where a $1/\sqrt{D}$ scaling was computed; here we extend the analysis to MSE and cross-entropy loss, which gives a $1/D$ scaling law.

If we let X be a random variable with V outcomes and probability distribution π , then $\hat{\pi}_a$ is the sample mean of the indicator function $I_a = \text{Ind}(X = a)$. The mean and variance of $\hat{\pi}_a$ are given by:

$$\begin{aligned} \mathbb{E}[\hat{\pi}_a] &= \pi_a \\ \text{Var}[\hat{\pi}_a] &= \mathbb{E}[(\hat{\pi}_a - \pi_a)^2] = \frac{1}{D} \pi_a (1 - \pi_a) \\ \mathbb{E}[(\hat{\pi}_a - \pi_a)^3] &= \mathbb{E}[\hat{\pi}_a^3 - 3\hat{\pi}_a^2 \pi_a + 3\hat{\pi}_a \pi_a^2 - \pi_a^3] = \frac{1}{D^2} \pi_a (2\pi_a - 1)(\pi_a - 1) \end{aligned} \quad (12)$$

Let us consider both mean-squared error and cross-entropy loss in this situation. By the central limit theorem, since $\hat{\pi}_a$ is the average over D i.i.d. binary random variables, its distribution in the large D limit is given by a Gaussian:

$$\hat{\pi}_a \sim \mathcal{N}(\pi_a, \frac{\sigma_a^2}{D}), \quad (13)$$

where $\sigma^2 = \pi_a(1 - \pi_a)$ is the variance of I_a .

E.1. Mean squared error

$$L_{MSE} = \frac{1}{|V|} \sum_{a \in V} (\pi_a - \hat{\pi}_a)^2. \quad (14)$$

Its expected value is

$$\begin{aligned}
 \mathbb{E}[L_{MSE}] &= \frac{1}{|V|} \sum_{a \in V} \mathbb{E}[(\hat{\pi}_a - \mathbb{E}[\hat{\pi}_a])^2] \\
 &= \frac{1}{|V|} \sum_{a \in V} \text{Var}[\hat{\pi}_a] \\
 &= \frac{1}{D} \frac{1}{|V|} \sum_{a \in V} \pi_a (1 - \pi_a)
 \end{aligned} \tag{15}$$

E.2. Cross-entropy loss

The cross-entropy loss is ²

$$L_{CSE} = - \sum_{a \in V} \pi_a \log \hat{\pi}_a \tag{16}$$

The expected value is:

$$\begin{aligned}
 \mathbb{E}[L_{CSE}] &= - \sum_{a \in V} \pi_a \mathbb{E}[\log \pi_a (1 + \frac{\hat{\pi}_a - \pi_a}{\pi_a})] \\
 &= S_\pi - \sum_{a \in V} \pi_a \sum_{n=1}^{\infty} \frac{(-1)^{(n+1)}}{n} \pi_a^{-n} \mathbb{E}[(\hat{\pi}_a - \pi_a)^n],
 \end{aligned} \tag{17}$$

where $S_\pi = - \sum_{a \in V} \pi_a \log \pi_a$ is the entropy of π . Evaluating the expectation values for $\hat{\pi}_a$ gives:

$$\begin{aligned}
 \mathbb{E}[L_{CSE}] &= S_\pi + \frac{1}{D} \frac{1}{2} \sum_{a \in V} (1 - \pi_a) - \frac{1}{3} \sum_a \frac{1}{\pi_a^2} \mathbb{E}[(\hat{\pi}_a - \pi_a)^3] + \dots \\
 &= S_\pi + \frac{|V| - 1}{2D} - \frac{1}{3D^2} \sum_a \frac{(2\pi_a - 1)(\pi_a - 1)}{\pi_a} + O(1/D^3) \\
 &= S_\pi + \frac{|V| - 1}{2D} + \frac{1}{D^2} (|V| - \frac{2}{3} - \frac{1}{3} \sum_a 1/\pi_a) + O(1/D^3)
 \end{aligned} \tag{18}$$

We see that in both cases, we get an exponent $\beta = 1$ for the asymptotic power-law scaling of the population loss. These results can easily be verified empirically.

E.3. Application to graph random walk

Let us consider applying the above to the graph random walk problem. There, for each node v , we want to learn a probability distribution $p(u|v)$ from the sequences. The node v itself appears with probability $p(v)$. Therefore if we have a dataset of D tokens, approximately $p(v)D$ of them will be v . So our effective dataset size for each node is $D_{\text{eff},v}$, which is equal to $p(v)D$ in the large D limit. For each node v we have a counting estimate $\hat{p}(u|v)$, leading to a cross-entropy loss for each node:

$$\mathbb{E}[L_{CSE,v}] = S_{p(\cdot|v)} + \frac{\deg(v) - 1}{2D_{\text{eff},v}} + O(1/D^2). \tag{19}$$

The total cross-entropy loss, averaging over all nodes is then:

$$\begin{aligned}
 \mathbb{E}[L_{CSE}] &= \sum_v p(v) \mathbb{E}[L_{CSE,v}] = \sum_v p(v) S_{p(\cdot|v)} + \sum_v \frac{\deg(v) - 1}{2D} + O(1/D^2) \\
 &= \langle S_{p(\cdot|v)} \rangle + \frac{2E - n}{2D} + O(1/D^2)
 \end{aligned} \tag{20}$$

For the unbiased random walk, $p(u|v) = A_{uv}/\deg(v)$ and the stationary distribution on nodes is $p(v) = \deg(v)/2E$.

²Note that for this to be well-defined for finite samples, where $\hat{\pi}_a$ might be zero even when $\pi_a \neq 0$, we must assume some smoothing, which we keep implicit.

F. Additional scaling law results

F.1. Number of bigrams vs. count: most bigrams appear only a few times

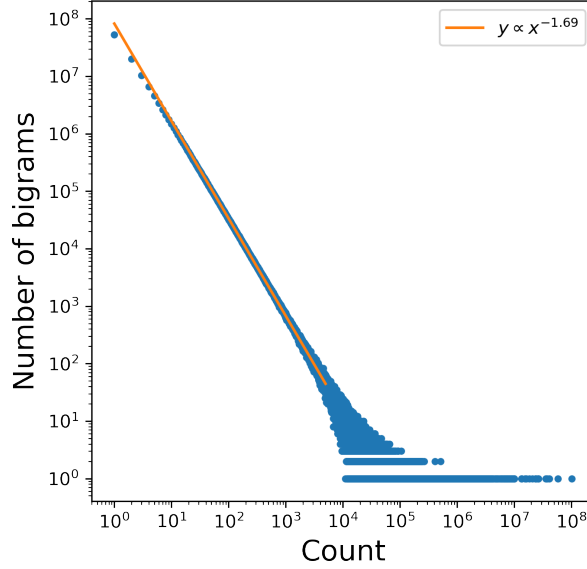


Figure 11. Number of bigrams that appear a given number (Count) of times, showing perfect linear fit on log-log scale. This plots demonstrates how almost all (over 100 million) bigrams appear only a handful of times.

F.2. Barabási-Albert graph with 8K nodes, 50K edges

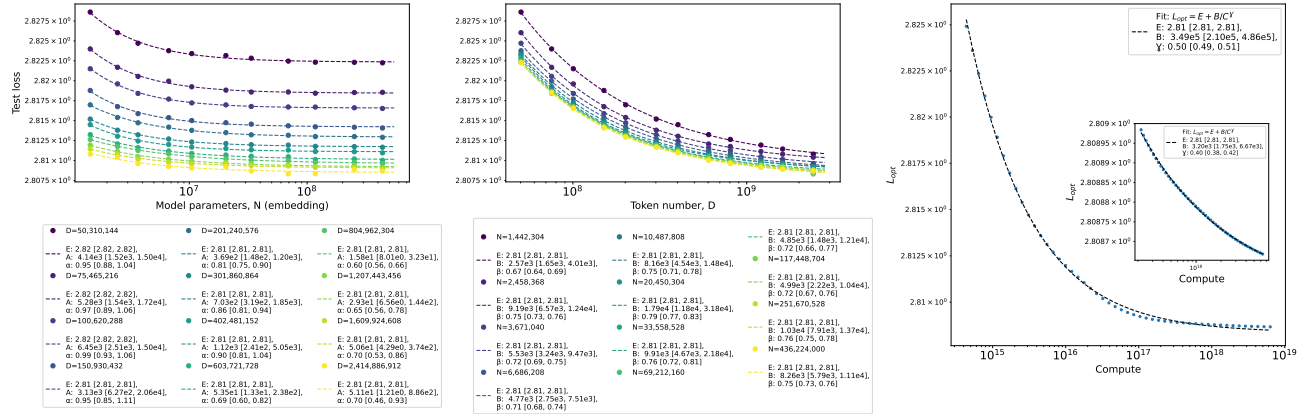


Figure 12. Results for Barabasi-Albert graph with 8,192 nodes. Mean $\overline{\alpha D} = 0.815$ with standard deviation 0.133. Mean $\overline{\beta_N} = 0.735$ with standard deviation 0.031. Average MSE for $L(N)_D$ 1d power law fits: 1.22×10^{-8} compared with 6.29×10^{-8} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits: 1.32×10^{-8} compared with 5.07×10^{-7} for best exponential fit.

F.3. Erdős-Renyi graphs with 1K nodes and 5K edges, $\kappa = 0$

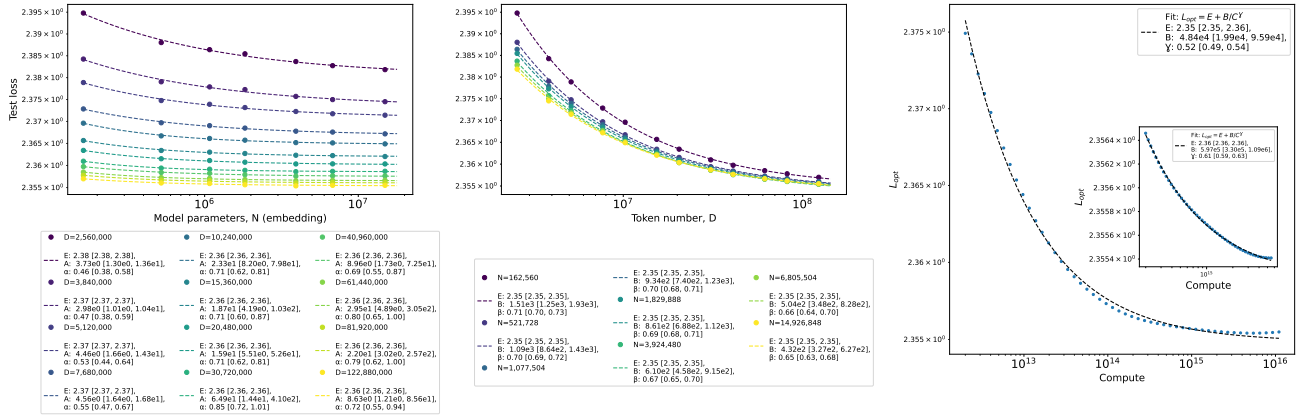


Figure 13. Scaling laws for unbiased ($\kappa = 0$) random walk on Erdos-Renyi graph with 1K nodes and 5K edges. Mean exponent $\bar{\alpha}_D = 0.665$ with standard deviation 0.125. Mean exponent $\bar{\beta}_D = 0.684$, with standard deviation 0.021. Average MSE for $L(N)_D$ 1d power law fits is $2.41e-08$, compared to 1.96×10^{-7} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits: 3.37×10^{-8} , compared to 1.97×10^{-6} for best exponential fit.

F.4. Erdős-Renyi graphs with 8K nodes and 50K edges, $\kappa = 1$

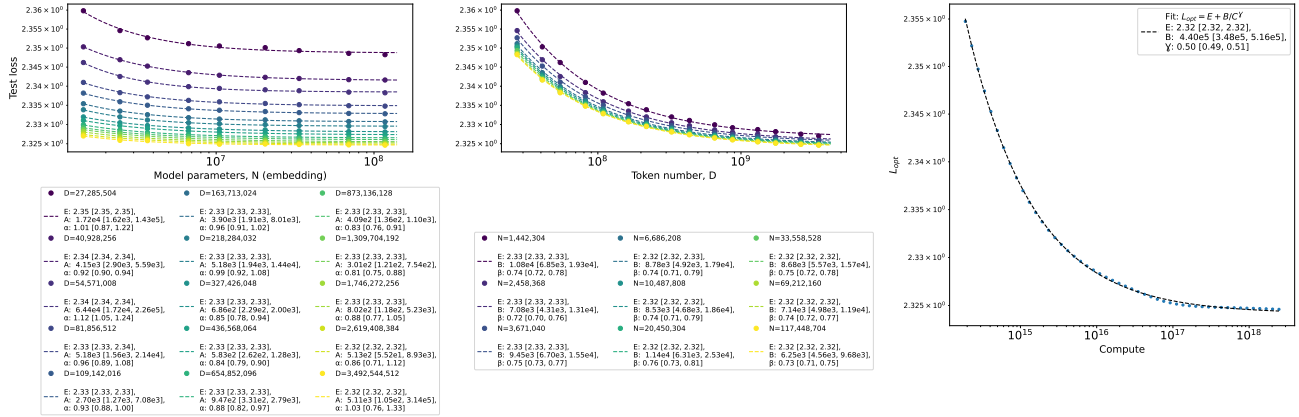


Figure 14. Scaling laws for Erdos-Renyi graph with 8K nodes and 50K edges, with power-law transition probabilities set by $\kappa = 1$. Mean exponent $\bar{\alpha}_D = 0.925$ with standard deviation 0.084. Mean exponent $\bar{\beta}_D = 0.741$, with standard deviation 0.010. Average MSE for $L(N)_D$ 1d power law fits is 1.76×10^{-8} , compared to 1.03×10^{-7} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits: 5.15×10^{-8} , compared to 2.12×10^{-6} for best exponential fit.

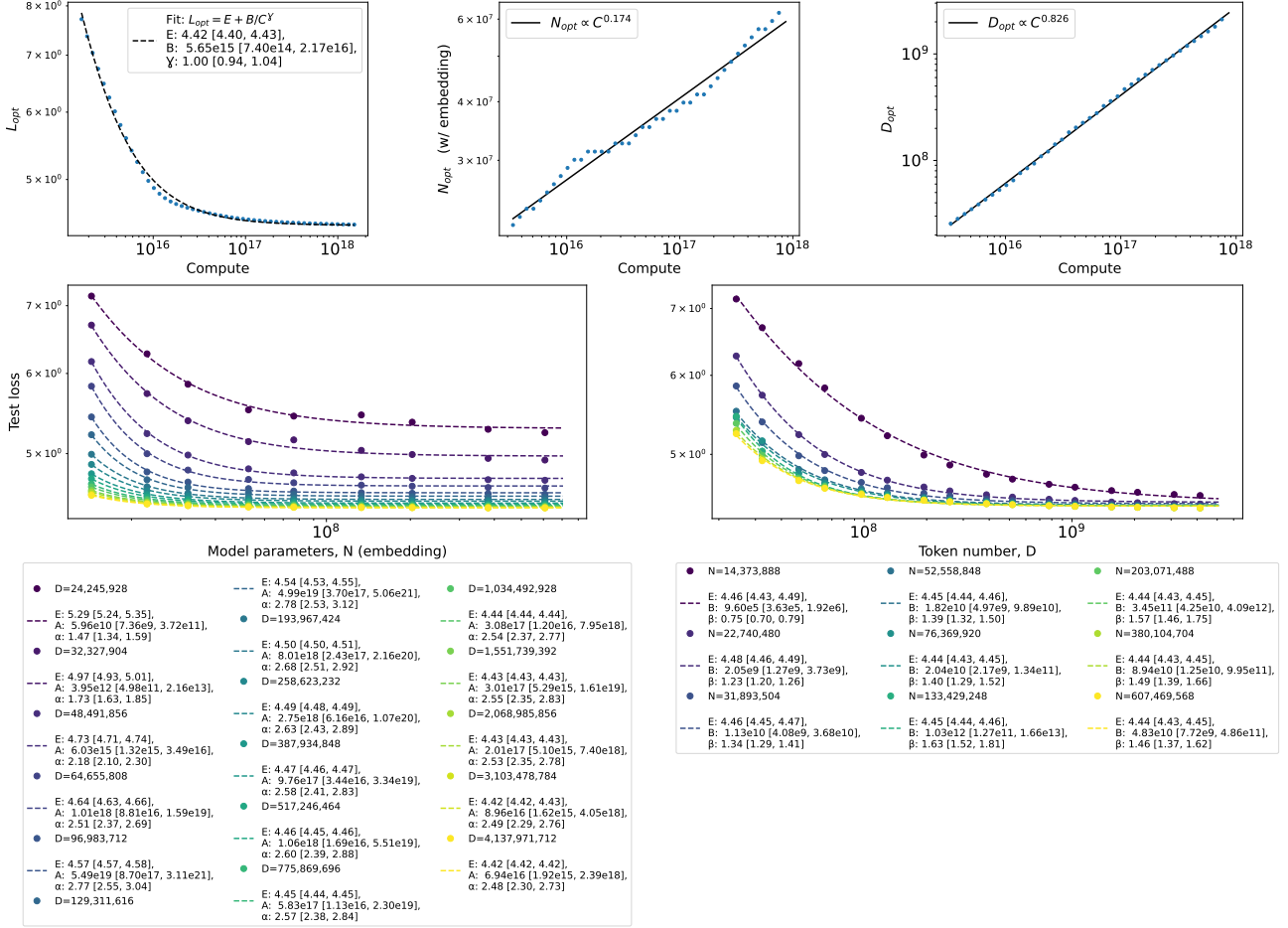
F.5. Erdős-Renyi graphs with 50K nodes and 2M edges, $\kappa = 0$


Figure 15. Results for Erdos-Renyi graph with 50,000 nodes and 2 million edges, with unbiased sampling $\kappa = 0$. Mean $\alpha = 2.443$, with standard deviation of 0.348. Mean $\beta = 1.361$ with standard deviation = 0.244. Average MSE for $L(N)_D$ 1d power law fits is 2.76×10^{-4} as compared with 8.23×10^{-4} for the best exponential fit. Average MSE for the $L(D)_N$ 1d power law fits is 3.99×10^{-4} , and best exponential fit gives 3.08×10^{-3} .

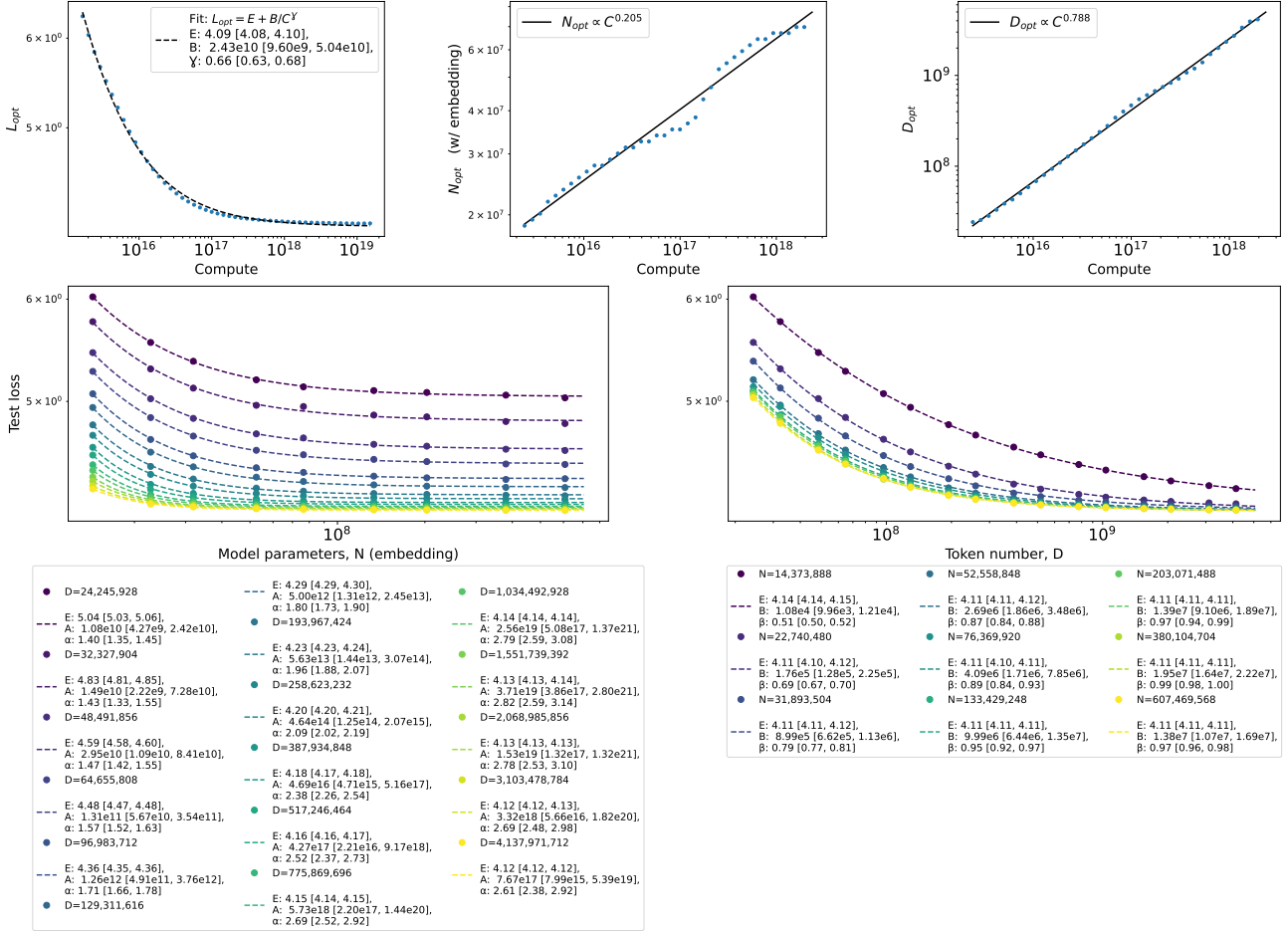
F.6. Erdős-Renyi graphs with 50K nodes and 2M edges, $\kappa = 1$


Figure 16. Results for Erdős-Renyi graph with 50,000 nodes and 2 million edges, with biased sampling $\kappa = 1$. Mean $\alpha = 2.169$ with standard deviation 0.529. Mean $\beta = 0.847$ with standard deviation 0.151. Average MSE for $L(N)_D$ 1d power law fits: 5.30×10^{-5} , compared with 4.25×10^{-4} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits: 4.05×10^{-5} , compared with 2.99×10^{-3} for best exponential fit.

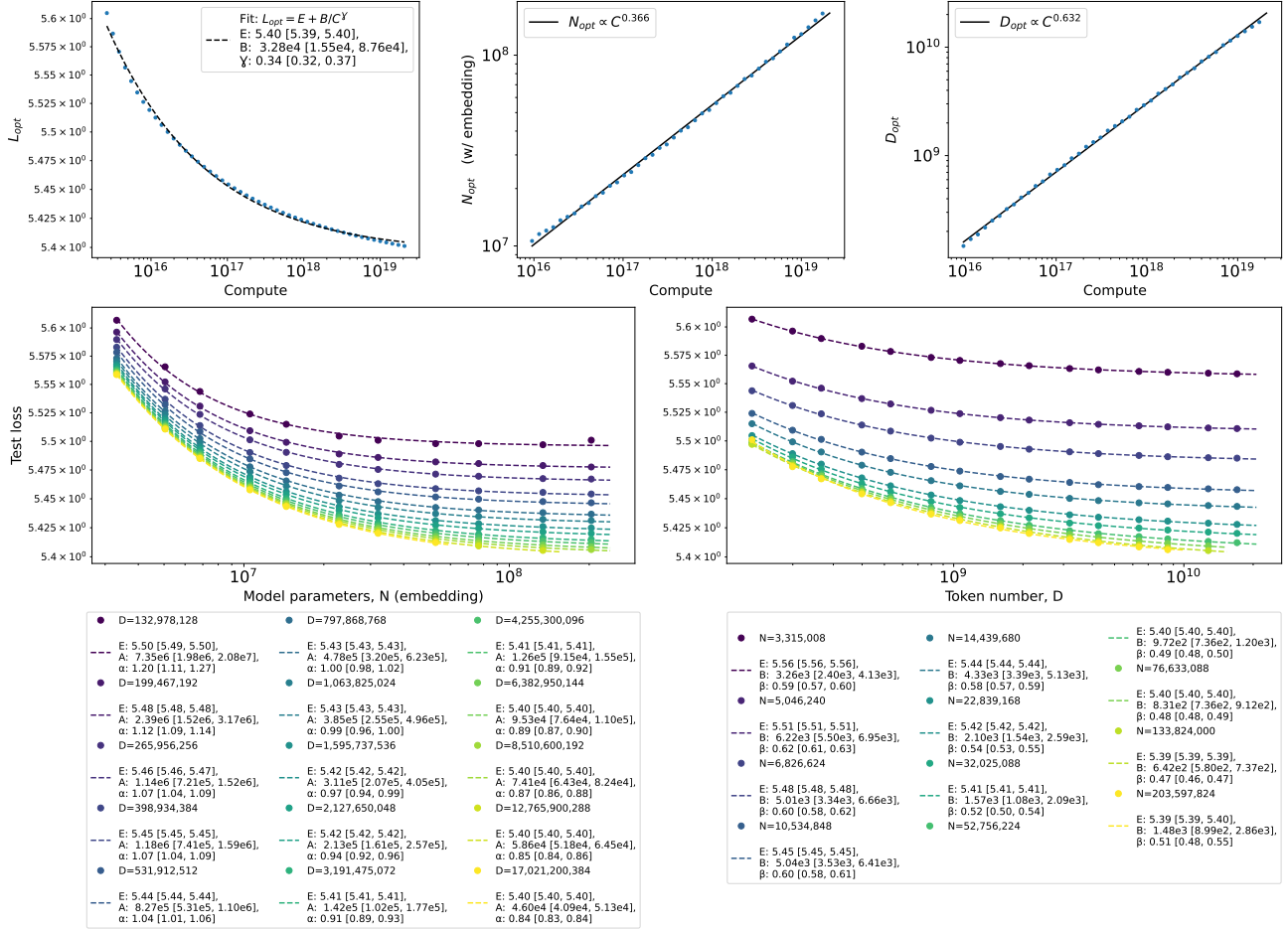
E7. 2 layer transformer trained on language bigram with μ Parametrization


Figure 17. 2 layer transformer trained on language bigram with μP . The α_D have a mean of $\overline{\alpha_D} = 0.977$ and a standard deviation of 0.102. The β_N have a mean of $\overline{\beta_N} = 0.547$ and a standard deviation of 0.052. Average MSE for $L(N)_D$ 1d power law fit is 5.22×10^{-7} , as compared to 2.91×10^{-5} for the best exponential fit. Average MSE for $L(D)_N$ 1d power law fits is 1.66×10^{-7} , as compared to 6.51×10^{-5} for the best exponential fit.

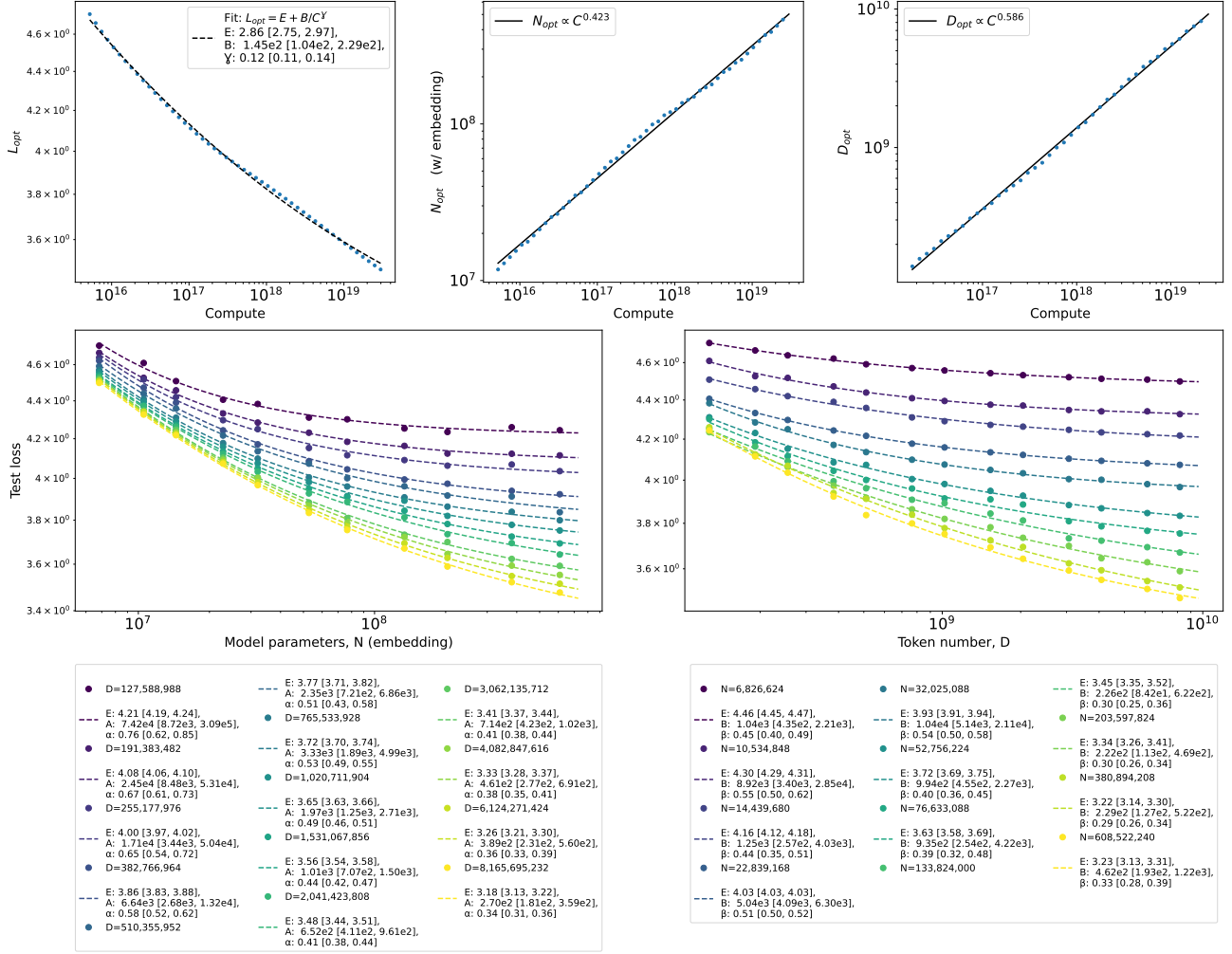
E.8. 2 layer transformer trained on language (Fineweb-edu) with μ Parametrization


Figure 18. 2 layer transformer trained on language (Fineweb-edu) with μP , context length 50, and batch size of 100 sequences. Mean exponent $\bar{\alpha}_D = 0.501$ with a standard deviation of 0.125. Mean exponent $\bar{\beta}_N = 0.408$ with standard deviation 0.093. Average MSE for $L(N)_D$ 1d power law fits is 1.19×10^{-4} , compared to 1.76×10^{-3} for the best exponential fits. Average MSE for $L(D)_N$ 1d power law fits is 9.05×10^{-5} , compared to 1.32×10^{-3} for the best exponential fits.

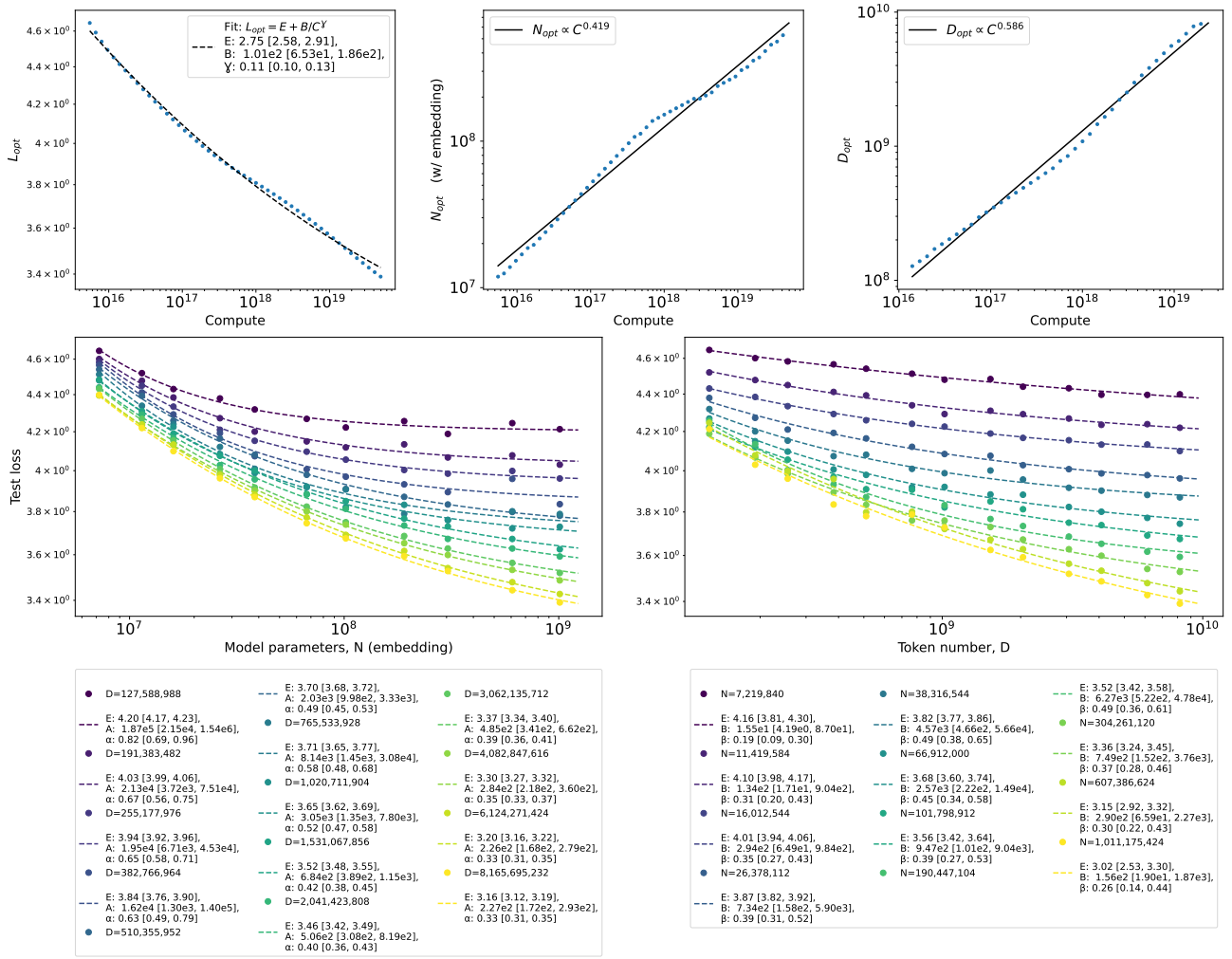
E9. 4 layer transformer trained on language (Fineweb-edu) with μ Parametrization


Figure 19. 4 layer transformer trained on language (Fineweb-edu) with μP , context length 50, and batch size of 100 sequences. Mean exponent $\bar{\alpha D} = 0.506$ with a standard deviation of 0.148. Mean exponent $\bar{\beta N} = 0.362$ with standard deviation 0.090. Average MSE for $L(N)_D$ 1d power law fits is 2.54×10^{-4} , compared to 2.30×10^{-3} for the best exponential fits. Average MSE for $L(D)_N$ 1d power law fits is 3.58×10^{-4} , compared to 2.04×10^{-3} for the best exponential fits.

F.10. 2 layer transformer trained on language (Fineweb-edu) with Standard Parameterization

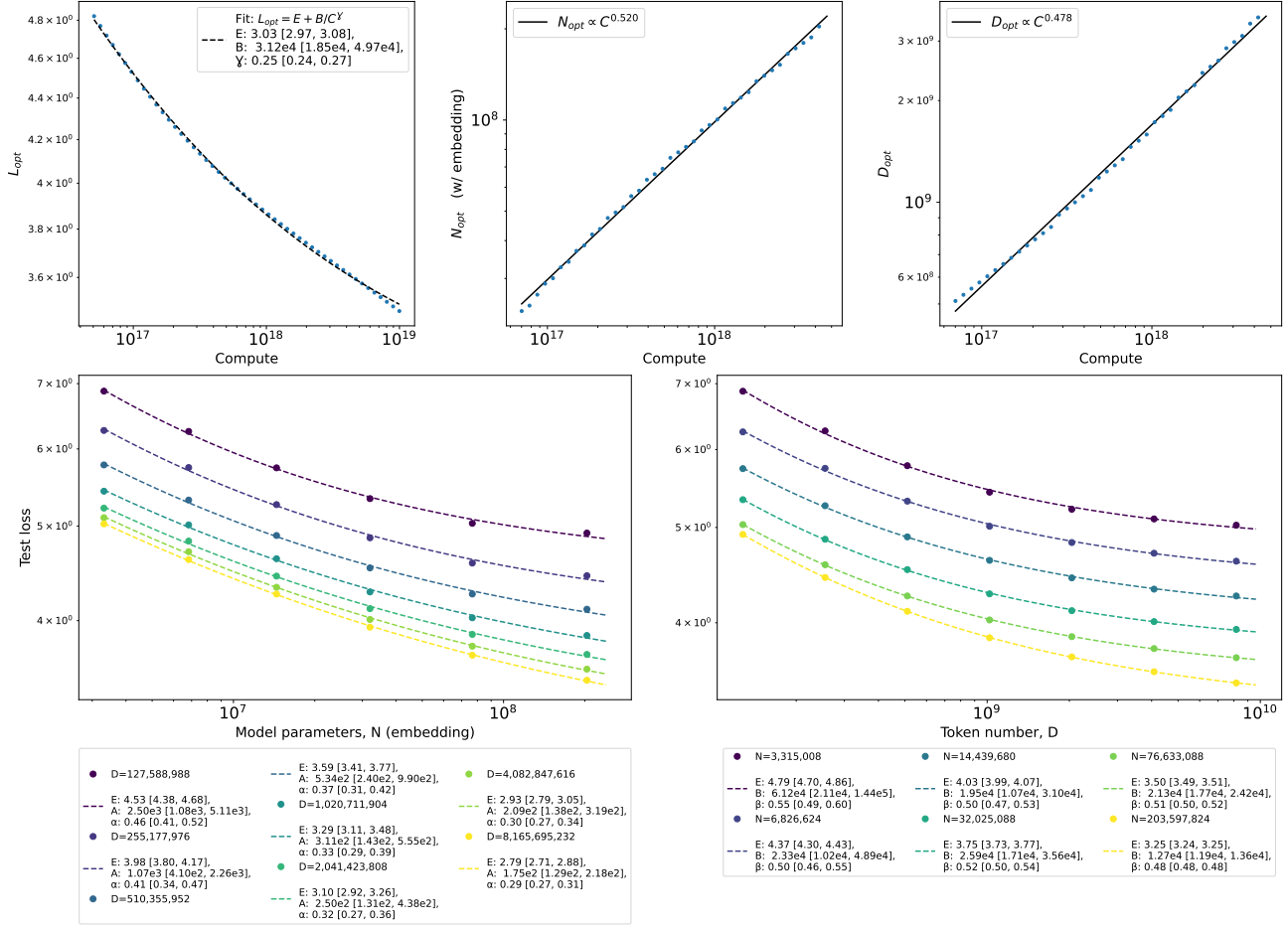


Figure 20. 2 layer transformer trained on language with SP , where N includes the embedding parameters. Mean exponent $\overline{\alpha_D} = 0.354$ with standard deviation 0.058. Mean exponent $\overline{\beta_N} = 0.511$ with standard deviation 0.022. Average MSE for $L(N)_D$ 1d power law fits is 5.41×10^{-4} . Average MSE for $L(N)_D$ 1d exp fits: 6.09×10^{-3} . Average MSE for $L(D)_N$ 1d power law fits: 1.88×10^{-4} . Average MSE for $L(D)_N$ 1d exponential fits: 1.00×10^{-2} .

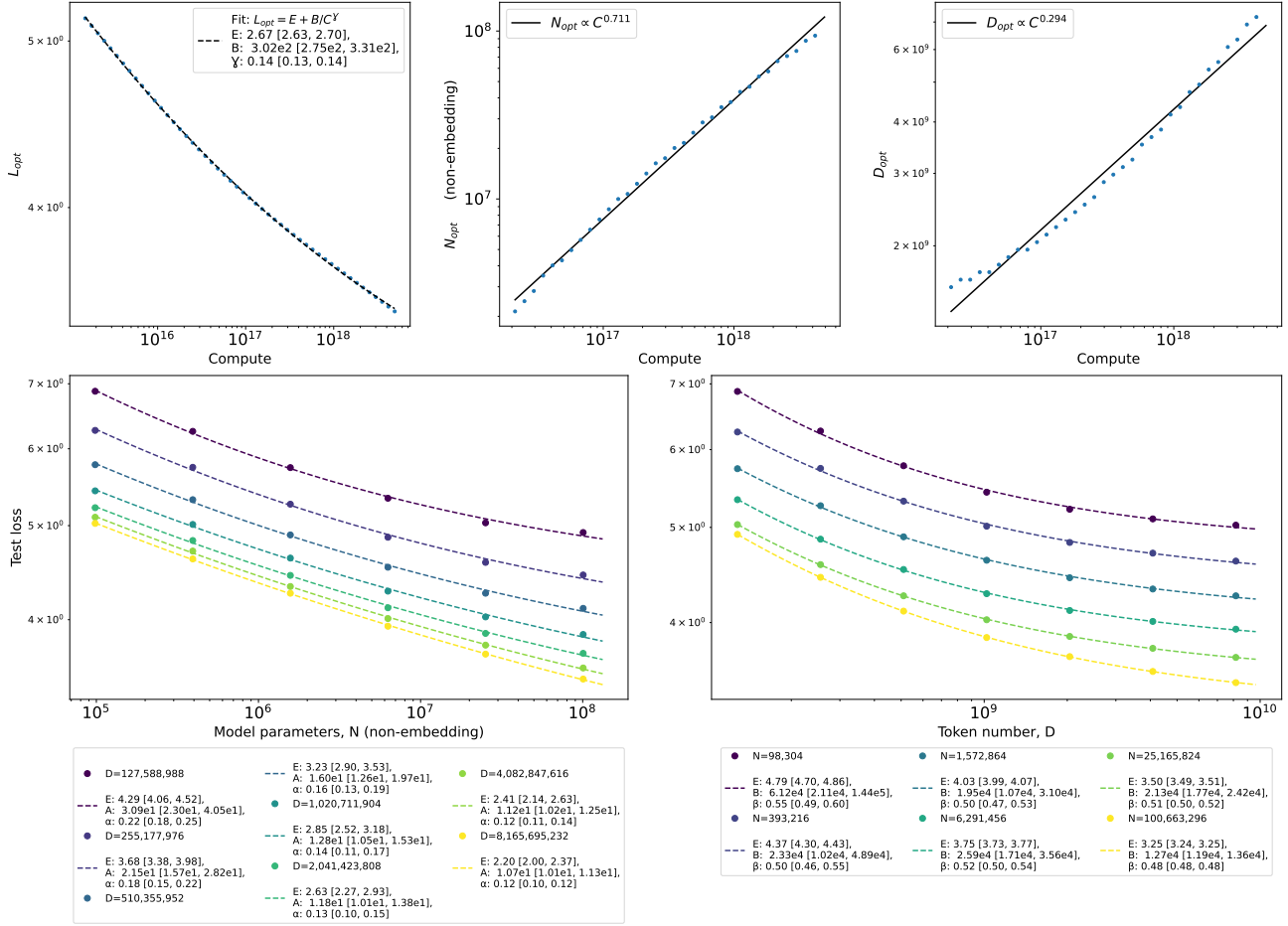
F.11. 2 layer transformer trained on language (Finweb-edu) with Standard Parameterization (and N equal to non-embedding parameters)


Figure 21. 2 layer transformer trained on language with SP. Here N refers to non-embedding parameters, in contrast to the rest of the paper.

F.12. 4 layer transformer trained on language (Fineweb-edu) with Standard Parameterization

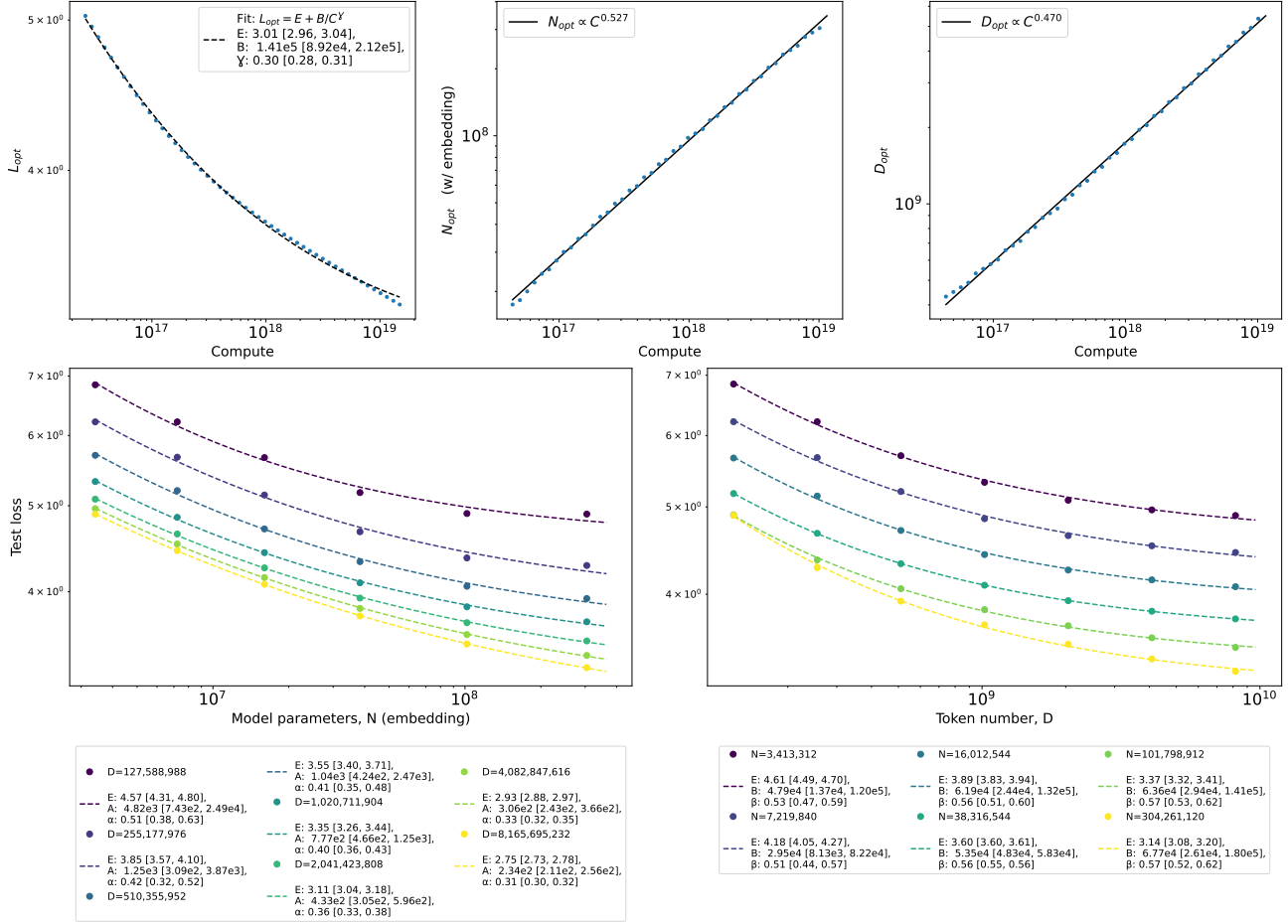


Figure 22. 4 layer transformer on language (Fineweb-edu) with Standard Parameterization. Mean exponent $\overline{\alpha_D} = 0.391$ with standard deviation 0.060. Mean exponent $\overline{\beta_N} = 0.550$ with standard deviation 0.020. Average MSE for $L(N)_D$ 1d power law fits is 1.28×10^{-3} , as compared with 7.11×10^{-3} for best exponential fit. Average MSE for $L(D)_N$ 1d power law fits is 4.51×10^{-4} , compared to 7.47×10^{-3} for best exponential fit.

F.13. 2 layer transformer trained on T1L

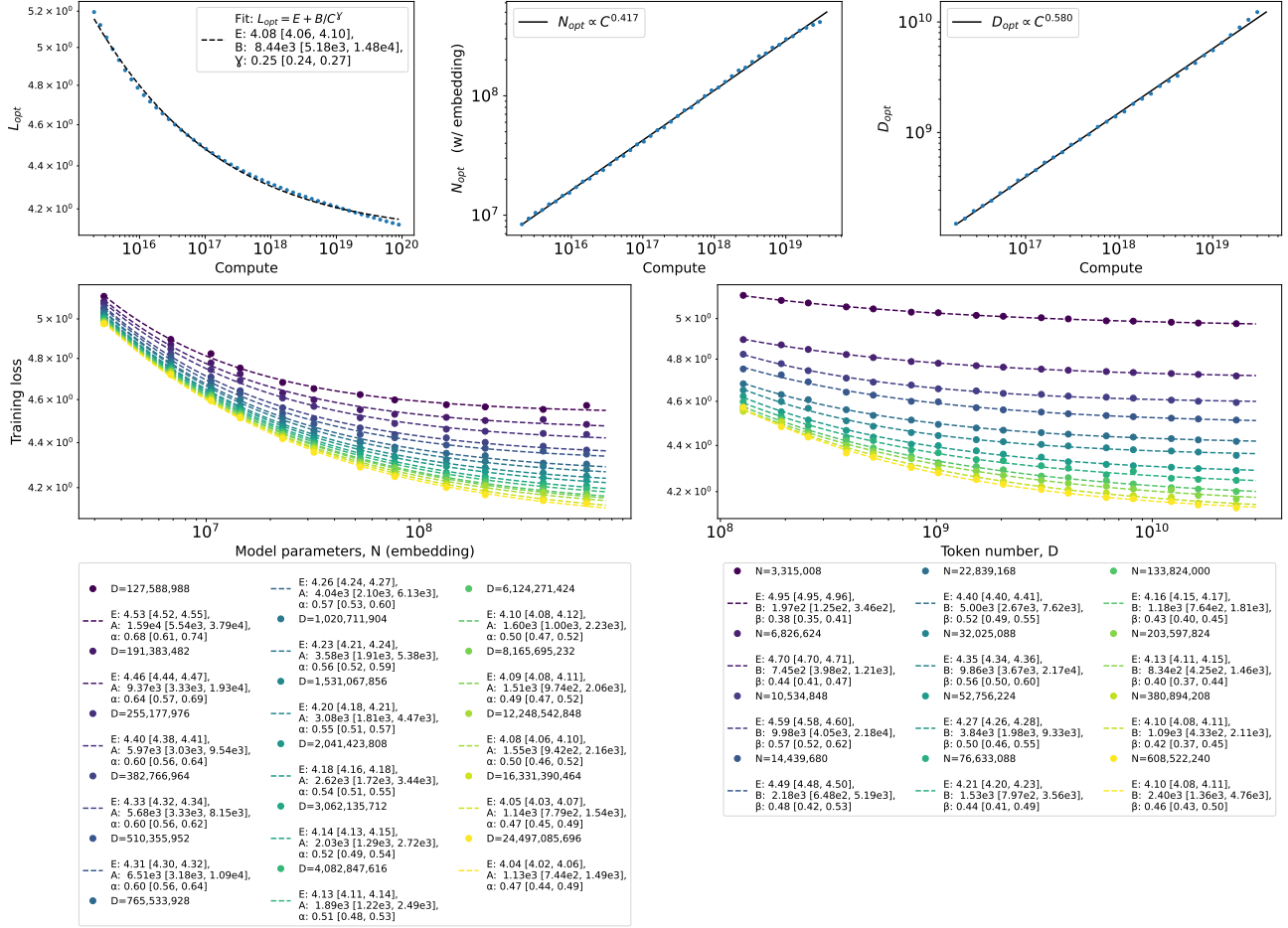


Figure 23. Scaling laws for 2 layer transformers learning T1L. The α_D have a mean of 0.550, with a standard deviation of 0.059. The β_N have a mean of 0.467, with standard deviation 0.058. The average MSE for the $L(N)_D$ 1d power law fits is 6.84×10^{-5} , while the average MSE for the best fit exponential fit is 1.49×10^{-3} . Average MSE for $L(D)_N$ 1d power law fits is 2.13×10^{-5} , compared to 1.27×10^{-3} for the best exponential fit. Note that training loss and test loss are equivalent in our setting of one-epoch training.

F.14. 2 layer transformer trained on T2L

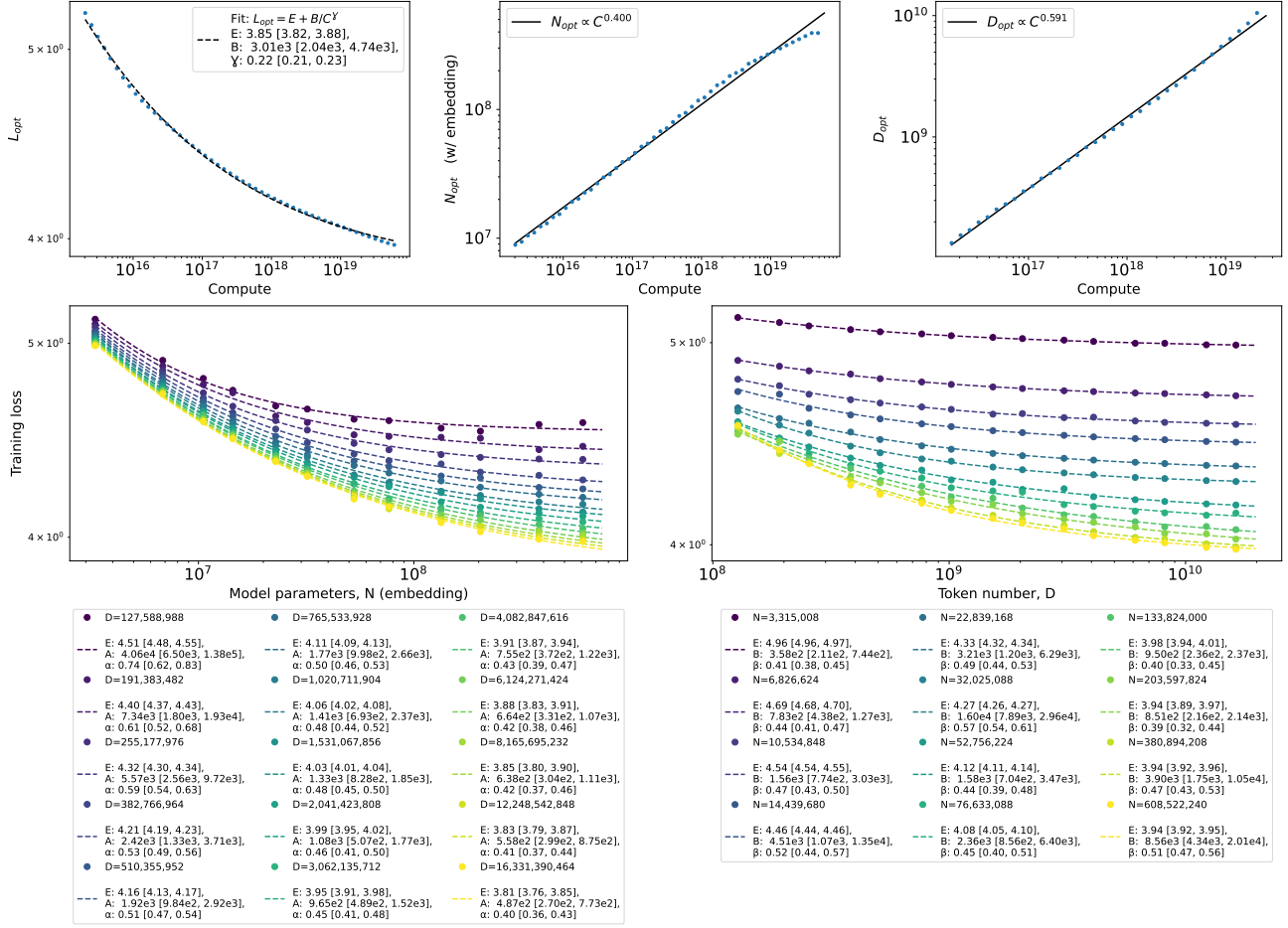


Figure 24. 2 layer transformer train on T2L. The mean α_D is 0.497, with standard deviation 0.088. The mean β_N is 0.463 with standard deviation 0.052. The average MSE for $L(N)_D$ 1d power law fits is 1.91×10^{-4} , compared to 2.03×10^{-3} for the best exponential fit. Average MSE for $L(D)_N$ 1d power law fits is 6.22×10^{-5} , compared to 1.52×10^{-3} for the best exponential fits. Note that training loss and test loss are equivalent in our setting of one-epoch training.